TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS

[Subject Code: EX 654]

A Minor Project Report
On
SOLAR BASED REAL TIME GREENHOUSE AUTOMATION SYSTEM

Submitted By:

ICCHYA BHATTARAI	PAS079BEI020
NANDITA TIWARI	PAS079BEI026
PRASHANT BHANDARI	PAS079BEI030
SUJAL SHRESTHA	PAS079BEI045

Submitted To:
Department of Electronics and Computer Engineering
Pashchimanchal Campus

May, 2026

LETTER OF APPROVAL

The undersigned certify that they have read and recommended to the Institute of Engineering for acceptance, a project report entitled **"Solar Based Real Time Greenhouse Automation System"** submitted by **Icchya Bhattarai, Nandita Tiwari, Prashant Bhandari, Sujal Shrestha** in partial fulfillment for the degree of Bachelor of Engineering in Electronics, Communication and Information Engineering..

Supervisor:
Asst. Professor Er. Hom Nath Tiwari
Department of Electronics and Computer Engineering

External Examiner

Asst. Professor Er. Hom Nath Tiwari
Head of Department, Department of Electronics and Computer Engineering

DATE OF APPROVAL: May, 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to our respected supervisor, **Assistant Professor Er. Hom Nath Tiwari**, for his invaluable guidance, continuous support, and encouragement throughout the course of this project. His insightful feedback and motivation were instrumental in shaping our understanding and execution of the project work.

We are equally thankful to the **Department of Electronics and Computer Engineering**, **Pashchimanchal Campus**, for providing the necessary resources, environment, and platform to carry out our minor project titled **"Solar Based Real Time Green- house Automation System"**.

We extend our heartfelt thanks to the **Head of Department, Er. Assistant Professor Er. Hom Nath Tiwari** , for his administrative support and encouragement. We would also like to acknowledge the contribution of the **Deputy Head of Department, Er. Ramesh Thapa**, for his guidance and facilitation during the project period.

Last but not least, we would also like to thank our friends and colleagues for their meaningful discussions, brainstorming sessions, and moral support. Your input has enriched the quality of this report and helped us navigate through challenging concepts

Icchya Bhattarai
Nandita Tiwari
Prashant Bhandari
Sujal Shrestha

ABSTRACT

"Solar Based Real Time Greenhouse Automation System" is a smart greenhouse automation system that monitors and controls key parameters of the greenhouse such as soil moisture, temperature and humidity to optimize plant growth. This system uses an ESP32 microcontroller as the central controller, the system integrates sensors like soil moisture and DHT for real time data collection. The automation includes a mini water pump for irrigation and a misting system controlled via mosfets, powered by a mini solar panel to promote energy efficiency. An OLED display provides local monitoring, while a backend system stores the collected data in a database and sends it to a user-friendly dashboard for remote monitoring and control. The system ensures high reliability with 99% voltage accuracy, achieved by sampling 2,500 data points per reading. Operating at 7.37 V with a current consumption of 0.08 A, the system has an approximate power consumption of 0.59 W, and it precisely monitors both solar energy intake and power usage. To save energy while maintaining real-time control, data is updated every one minute, balancing consistent monitoring with long-term power efficiency. This solution reduces the need for manual work while improving how water and energy are used, making greenhouse farming more practical, efficient, and sustainable.

Keywords : *Automation,ESP-32, Monitoring, BLE, Solar-Powered*

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL i

ACKNOWLEDGEMENT ii

ABSTRACT iii

LIST OF FIGURES vi

LIST OF TABLES vii

List of Abbreviations viii

1 INTRODUCTION 1

1.1 Background 1

1.2 Problem statement 2

1.3 Objectives 2

2 LITERATURE REVIEW 3

3 REQUIREMENTS 5

3.1 Hardware Requirements 5

3.2 Software Requirements 11

4 METHODOLOGY 13

4.1 Block Diagram 13

4.2 Flow Chart 15

4.3 Hardware Setup and System Design 16

4.3.1 Sensor Node (Data Acquisition Unit) 16

4.3.2 Control Unit (Processing and Actuation Unit) 16

4.3.3 Power Management Unit (Solar Charging System) 17

4.3.4 Power Budget Analysis 19

4.3.5 Experimental Power and Energy Calculation (Measured Data) 20

4.3.6 Battery Voltage Monitoring and Calibration 21

4.3.7 Physical Arrangement of the System 26

4.3.8 Pin Configuration and Peripheral Interface 28

4.4 Data Acquisition and Transmission 29

4.5 Data Processing and Monitoring 30

4.5.1 Real-Time Data Handling 30

4.5.2 Data Representation 30
4.5.3 Remote Monitoring via BLE 31
4.6 Automation Logic and Threshold Calibration 31
4.7 Sensor Data Filtering and Noise Reduction 34
4.8 BLE-Based Monitoring and Data Management System 36
4.8.1 BLE Communication Architecture 36
4.8.2 User Monitoring Interface 37
4.8.3 Data Handling and Logging 37
4.8.4 System Data Flow Architecture 38
4.8.5 Monitoring Capability 38
4.8.6 Performance Analysis 38
4.9 Software System Design 39
4.9.1 System Architecture and Operation 40
4.9.2 Firmware and BLE Communication 40
4.9.3 Backend, Frontend, and System Flow 41
4.9.4 Database, API, and Deployment 43

5 EPILOGUE **44**
5.1 RESULTS AND FINDINGS 44
5.1.1 Relation in Auto and Manual Mode 44
5.1.2 Environmental Time Series 48
5.1.3 Solar Charging and Energy Performance Analysis 51
5.1.4 Detection and System Performance 53
5.1.5 Key Findings 53
5.2 LIMITATIONS AND FUTURE ENHANCEMENTS 54
5.2.1 LIMITATIONS 54
5.2.2 FUTURE ENHANCEMENTS 54
5.3 CONCLUSION 55

REFERENCES **57**

List of Figures

3.1 Solar Panel 5
3.2 Li-Po 3.7V Pouch Battery 6
3.3 ESP32 Board 6
3.4 DHT11 Sensor 7
3.5 Soil Moisture Sensor 8
3.6 DC Water Pump 8
3.7 Ultrasonic Mist Maker 9
3.8 OLED Display Board 9
3.9 TP4056 Charging Module 10
3.10 MP1584 Voltage Regulator 10
3.11 IRLZ44N MOSFET 11
4.1 Block Diagram 13
4.2 Flow Chart of Complete Workflow 15
4.3 DC Pump circuit 17
4.4 Mist Maker circuit 17
4.5 Schematic of Voltage Regulator 18
4.6 Schematic of Voltage Divider 22
4.7 Physical Arrangement of Greenhouse 27
4.8 ESP32 Pin Configuration 29
4.9 RSSI Chart 39
4.10 Frontend Dashboard 42
5.1 Correlation analysis in AUTO Mode 44
5.2 Correlation analysis in MANUAL Mode 46
5.3 Environmental Time Series 48
5.4 Activation Regions of Mist Maker 49
5.5 Motor Pump ON/OFF 50
5.6 Solar Current vs Charging Current 51
5.7 Battery Voltage Variation 52
5.8 Solar Panel Voltage Variation 52

List of Tables

4.1 System Power Consumption Table 19
4.2 MOSFET Power Loss for Switching Loads 19
4.3 ESP32 Pin Configuration and Peripheral Interface 29
5.1 Performance Comparison Between AUTO and MANUAL Operating Modes 47
5.2 Detection and system performance metrics 53

List of Abbreviations

ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
API	Application Programming Interface
BLE	Bluetooth Low Energy
BMS	Battery Management System
DC	Direct Current
GPIO	General Purpose Input/Output
IoT	Internet of Things
LiPo	Lithium Polymer
LoRa	Long Range
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MPPT	Maximum Power Point Tracking
OLED	Organic Light Emitting Diode
PWM	Pulse Width Modulation
SoC	System on a Chip

1. INTRODUCTION

1.1. Background

Nepal is an agricultural country. Here, the majority of people are dependent on agriculture for their livelihood. But agricultural productivity here is comparatively less due to factors like complex geographical conditions, limited resources and facilities, and increasing climate variations. Many farmers still rely on seasonal crops that are affected by temperature changes, irregular rainfall, and extreme weather. These challenges are more in rural and mountainous regions, where access to modern agricultural methods and reliable energy sources is limited. Lately, greenhouse farming is emerging as a solution to all these issues. However, in Nepal, greenhouse management is still performed manually, requiring continuous human involvement to monitor temperature, humidity, soil moisture, light intensity and every other factor. This is time consuming and not so accurate, especially for those farmers who may not be able to constantly supervise the greenhouse throughout the day.

Recently with the advancements in automation and related technologies, it is possible to monitor and control greenhouse environments in real time. But, most existing automated systems depend on electricity, which is usually unreliable or unavailable in many rural areas of Nepal. Power shortages and inconsistent supply further limit the scope of such technologies.

Nepal has significant potential for solar energy so integrating the solar power with greenhouse automation offers a sustainable and reliable solution for remote and off-grid agricultural settings. A solar based real time greenhouse automation system can ensure continuous operation without depending on the conventional power sources.

This project focuses on the design and implementation of a solar powered, real time greenhouse automation system that monitors key environmental parameters of a greenhouse i.e, temperature, humidity and soil moisture. The system will automatically control the mentioned greenhouse conditions using an ESP32 and display the real time data in the greenhouse as well as dashboard. By combining the system with solar energy, the proposed system targets to improve agricultural productivity, reduce manual labor, and support sustainable farming practices suitable for Nepal.

1.2. Problem statement

Traditional greenhouse farming in Nepal is majorly based in manual labour for temperature regulation and irrigation, which is physically demanding and highly susceptible to error. Farmers in Nepal's diverse topographical regions lack the real-time data and automated tools necessary to respond to sudden environmental changes. Although automated greenhouse systems exist, they are often costly, complicated, and depend on a continuous power supply, which is unreliable in many rural and semi rural areas.While solar power offers a sustainable energy solution for these remote settings, the primary challenge remains the lack of integrated systems that can independently monitor and adjust internal conditions. There is a critical need for a low-cost automation framework that autonomously manages temperature, humidity, and soil moisture. By utilizing solar assisted power to drive these automated responses, such a system would eliminate the constant need for manual monitoring, stabilize crop yields, and modernize agricultural practices in areas where technical resources are limited.

1.3. Objectives

To develop an automated, real-time, solar-powered greenhouse monitoring system.

2. LITERATURE REVIEW

Greenhouse automation systems have gained significant attention as an efficient and sustainable alternative to conventional greenhouse management methods.The goal of this review is to present a comprehensive summary of the existing work in greenhouse automation, with a focus on various approaches, technologies, and their effectiveness in optimizing environmental conditions such as temperature, humidity, and soil moisture.

Candy et al. [1] studied the design, construction, and thermal performance of solar greenhouses in Humla, Nepal, highlighting how high-altitude conditions and limited resources affect crop production. Their work demonstrated the effectiveness of insulated walls, double-layer glazing, and thermal mass in maintaining suitable air and soil temperatures for winter crops. This study informed structural and material choices for greenhouse design and showed the potential for improving crop yield in cold regions. However, the system relied on manual ventilation and lacked real-time monitoring and automation, which limits efficiency and responsiveness to environmental changes. The proposed system addresses these limitations through IoT-based real-time monitoring and automated climate control.

Siddiqui et al. [2] developed a smart greenhouse system using sensors and a microcontroller to monitor temperature, humidity, soil moisture, and light intensity, while actuators automatically adjust environmental conditions. The system enables real-time remote monitoring through web and mobile applications, allowing farmers to manage greenhouse conditions without physical presence. Their approach demonstrates how IoT-based automation can optimize microclimatic conditions, improve crop yield, and reduce manual labor, making it suitable for small-scale and resource-constrained farming environments.

Abou-Mehdi-Hassani et al. [3] conducted a systematic review of smart greenhouse monitoring systems, emphasizing the role of IoT in real-time data collection and automated climate control. The study analyzed 54 research papers published between 2019 and 2023, showing that integration of IoT with artificial intelligence, machine learning, and fuzzy logic significantly improves resource efficiency and crop productivity. The authors also highlighted the importance of energy-efficient and solar-powered systems for sustainable greenhouse operations, particularly in remote agricultural areas.

Omid and Shafaei [4] investigated temperature and relative humidity variations inside greenhouses compared to external conditions using multiple sensors. Their

results showed that internal temperatures are consistently higher due to trapped solar radiation, and vertical temperature gradients exist within the structure. They also demonstrated that automated control systems using fans and heaters can maintain stable internal conditions, highlighting the importance of continuous environmental monitoring for effective greenhouse management.

Ayuba [5] implemented a digital measurement system using the DHT11 sensor to monitor temperature and humidity. The study demonstrated how the sensor integrates a thermistor and capacitive humidity element to provide a digital output to a microcontroller via a single-wire interface. This simplifies system design and reduces the need for complex analog processing. The findings show that low-cost sensors like DHT11 can effectively support real-time environmental monitoring in automated systems.

Singh [6] evaluated the application of solar photovoltaic systems in the powering of automated agricultural infrastructure. The study explains how solar energy is converted into electrical energy using the photovoltaic effect and highlights its use in the powering of sensors, irrigation pumps, and control systems. The research concludes that solar-based systems provide a sustainable and reliable energy source for remote agricultural applications, reducing dependency on conventional grid electricity while ensuring continuous operation of automated systems.

3. REQUIREMENTS

The various tools used for the implementation of this project are listed below:

3.1. Hardware Requirements

1. **Solar Panel**

   Thin-film solar cells are a type of solar cell made by depositing thin layers of photovoltaic material onto a glass, plastic or metal substrate. Polycrystalline silicon based solar cells show energy conversion efficiencies of 8% and high manufacturing yields of $> 90\%$. **[7]**

   A solar panel converts sunlight into DC electricity using the photovoltaic effect. When sunlight falls on the panel, it releases electrons and produces an electric current.

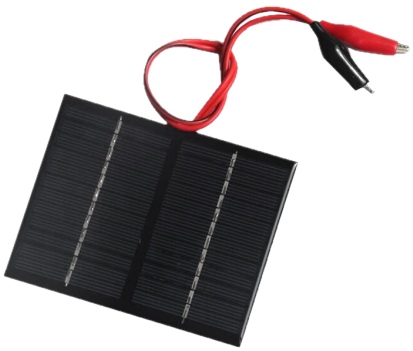

Figure 3.1: Solar Panel

2. **Li-Po Battery**

   LiPo batteries, short for lithium polymer batteries, are a special type of lithium-ion batteries that offer improved performance and flexibility. These batteries use a solid or gel-like polymer electrolyte instead of the liquid electrolytes found in conventional lithium-ion batteries. A LiPo battery consists of several main components: an anode, a cathode, a polymer electrolyte, and a separator. **[8]**

   It acts as a reservoir for electricity generated by solar panels, along with a charge controller which regulates it to ensure safety.

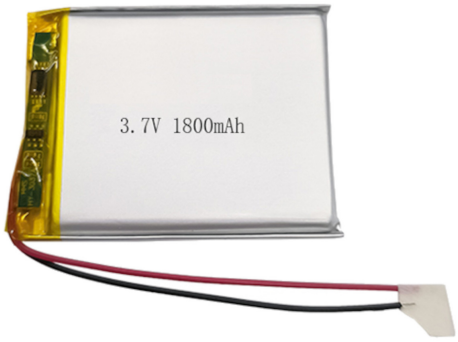

Figure 3.2: Li-Po 3.7V Pouch Battery

3. **ESP32**

   ESP32 is a series of low-cost and low-power System on a Chip (SoC) microcontrollers developed by Espressif that include Wi-Fi and Bluetooth wireless capabilities and dual-core processor. **[9]**

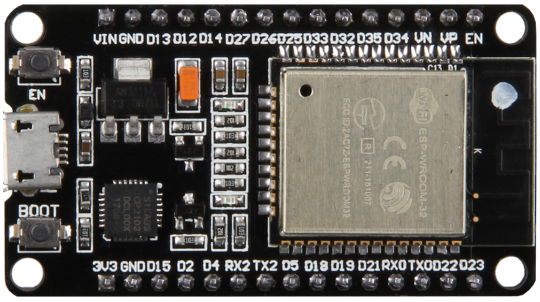

Figure 3.3: ESP32 Board

4. **DHT11 Sensor**

   DHT11 Temperature & Humidity Sensor features a temperature & humidity sensor complex with a calibrated digital signal output. This sensor includes a resistive-type humidity measurement component and an NTC temperature

measurement component, and connects to a high performance 8-bit microcontroller. **[10]**

The DHT11 sensor provides relative humidity value in percentage (20 to 90% RH) and temperature values in degree Celsius (0 to 50 °C).

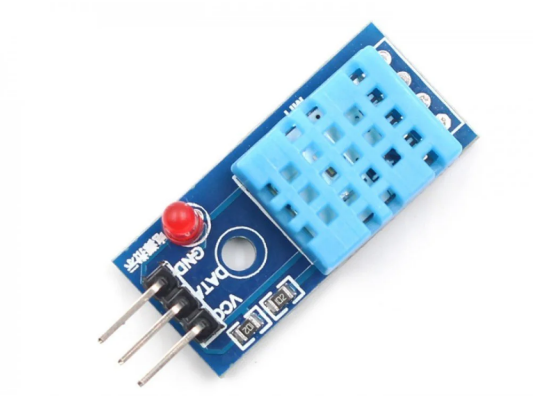

Figure 3.4: DHT11 Sensor

5. **Soil Moisture Sensor**

   The soil moisture sensor or the hygrometer is usually used to detect the humidity of the soil. So, it is perfect to build an automatic watering system or to monitor the soil moisture of your plants. The sensor is set up by two pieces: the electronic board and the sensor probe with two pads, that detects the water content. The output can be an analog signal and so get a value between 0 and 4096. **[11]**

   It works based on changes in the soil's electrical properties (either resistance or capacitance). In resistive sensors, higher moisture reduces electrical resistance between the probes due to increased conductivity. In capacitive sensors, the dielectric constant of the soil changes with water content, altering the capacitance, which is then converted into a measurable signal.

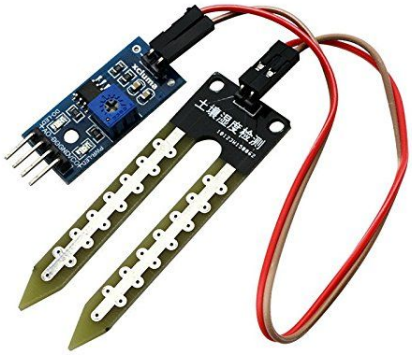

Figure 3.5: Soil Moisture Sensor

6. **DC Water Pump**

   A water pump is a mechanical device designed to move water from one location to another, utilizing pressure changes within a system. The pump's motor, which can be electric (AC or DC) or gas-powered, drives this process. Water enters through an inlet valve or pipe, moves through the impeller chamber, and exits through an outlet valve or pipe. **[12]**

Figure 3.6: DC Water Pump

7. **Mist Maker**

   The ultrasonic mist fogger, also known as ultrasonic mister/fogger, is a relatively small device which is commonly placed in a container filled with water,

and the high-frequency ultrasonic oscillator can break water into countless water droplets which are sprayed into air as a fog-like cloud. **[13]**

Figure 3.7: Ultrasonic Mist Maker

8. **OLED Display**

   OLED (Organic Light Emitting Diodes) is a flat light emitting technology, made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. OLEDs are emissive displays that do not require a backlight and so are thinner and more efficient than LCD displays. **[14]**

   OLED displays commonly use the I2C protocol for data communication, requiring only SDA and SCL lines.

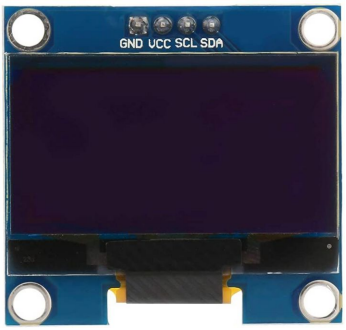

Figure 3.8: OLED Display Board

9. **TP4056**

The TP4056 is a complete constant-current/constant-voltage linear charger for single cell lithium-ion batteries. It is suitable for portable applications and automatically terminates charging when current drops to 1/10th of programmed value. **[15]**

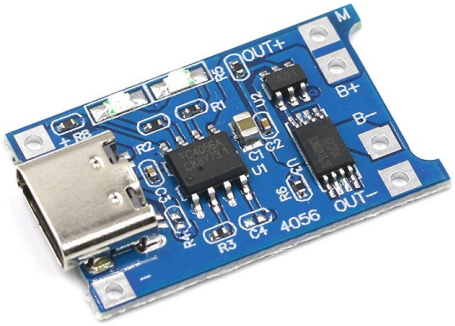

Figure 3.9: TP4056 Charging Module

10. **MP1584**

The MP1584 is a high frequency step-down switching regulator with integrated MOSFET providing 3A output capability and efficient power conversion. **[16]**

Figure 3.10: MP1584 Voltage Regulator

11. **IRLZ44N MOSFET**

The IRLZ44N is an N-channel MOSFET with low resistance and fast switching capability. It supports high current applications with gate threshold voltage of 1–2V. **[17]**

Figure 3.11: IRLZ44N MOSFET

3.2. Software Requirements

1. **Platform IO**

   PlatformIO is a cross-platform, cross-architecture, multiple framework, professional tool for embedded systems engineers and for software developers who write applications for embedded products. **[18]**

   It handles firmware development, library management, and uploading code to the ESP32.

2. **KiCad**

   KiCad is a free and open-source electronics design automation (EDA) suite. It features schematic capture, integrated circuit simulation, printed circuit board (PCB) layout, 3D rendering, and plotting/data export to numerous formats. KiCad also includes a high-quality component library featuring thousands of symbols, footprints, and 3D models. **[19]**

3. **Bluetooth Low Energy (BLE)**

   Bluetooth Low Energy is a communication protocol. And just like any other protocol, it acts as a rulebook that devices must adhere to in order to communicate or exchange information. The Bluetooth protocol stack is broadly broken down into three primary components or subsystems i.e. application, host, and controller. Inside each of these blocks are distinct layers. **[20]**

BLE can cover up to 100 meters, but the exact distance varies depending on the environment and implementation.**[21]** It is preferred as it offers lower power consumption and is well-suited for our required setup. Data packets containing humidity, temperature and moisture data are sent at regular intervals to the backend and data is stored.

4. **Backend System**

   The backend is implemented using Node.js with the NestJS framework, providing a modular and scalable architecture for REST API services. It handles data ingestion from the BLE gateway, validates incoming sensor data, manages system logic, and coordinates actuator control through command handling.

   The backend also supports alert generation, system state management, and communication with the database layer. Structured APIs enable interaction with the frontend dashboard, while internal services ensure reliable processing and routing of sensor data and control commands.

5. **Frontend System**

   The user interface is developed using Next.js with React and TypeScript, providing a modern and responsive web-based dashboard for real-time monitoring and control.

   The interface presents environmental parameters such as temperature, humidity, soil moisture, and actuator states through dynamic components. Styling is handled using Tailwind CSS, while Recharts is used for visualizing historical and real-time data trends.

   Efficient data fetching, caching, and synchronization are managed using TanStack Query, enabling near real-time updates without requiring continuous page reloads.

6. **Database System**

   The system uses PostgreSQL as the primary relational database for persistent data storage. It maintains structured records of sensor readings, actuator states, user information, alerts, and system configurations.

   Database interaction is handled through Prisma ORM, which provides type-safe queries, schema management, and efficient data access. Referential integrity is maintained across related entities, ensuring consistency and reliability of stored data.

4. METHODOLOGY

4.1. Block Diagram

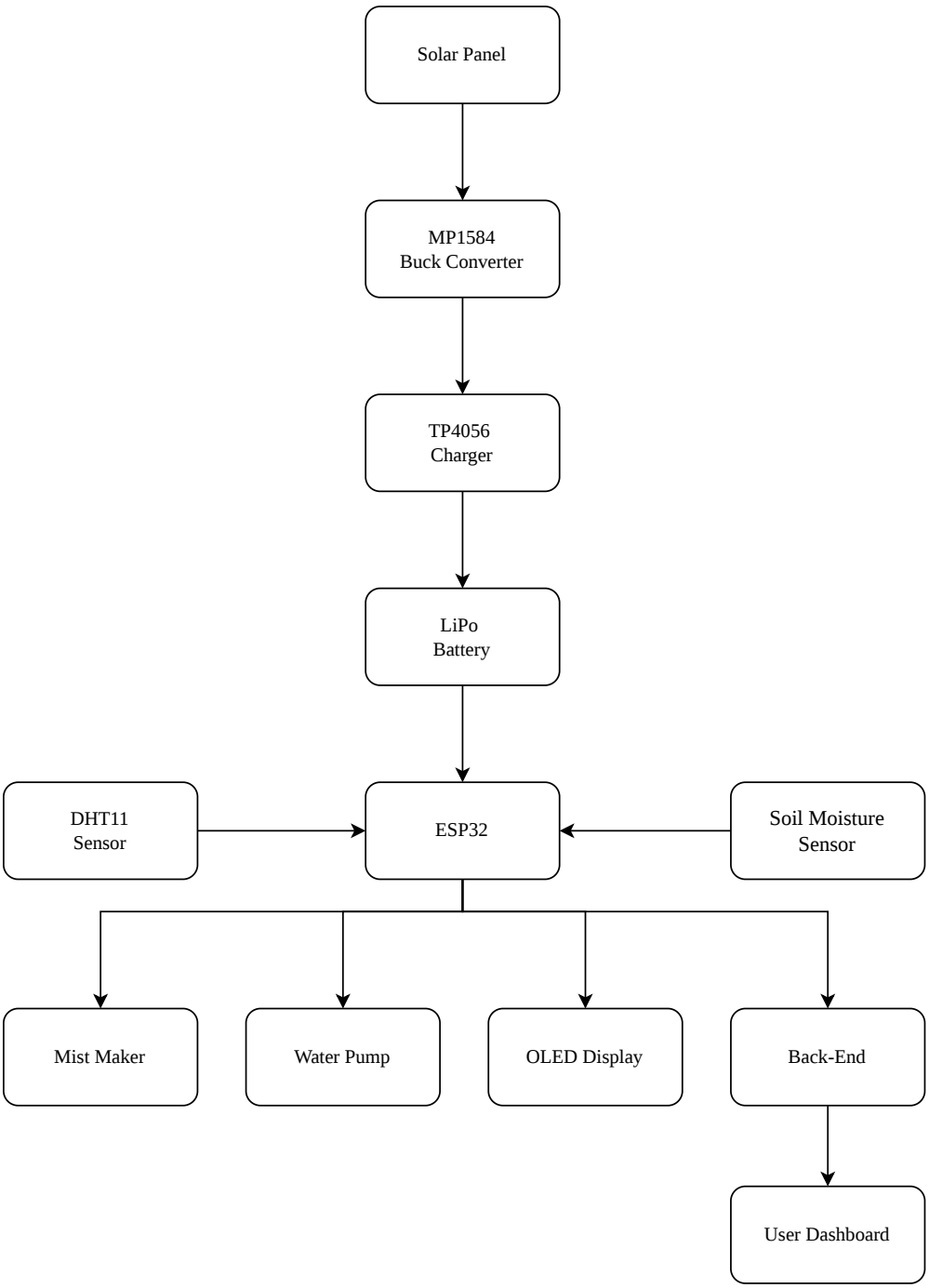

Figure 4.1: Block Diagram

The figure illustrates the overall architecture and working principle of the greenhouse monitoring and automation system. The system is designed to provide automated environmental control, real-time monitoring, and efficient power management using solar energy.

The power section consists of a solar panel that converts solar energy into electrical energy. The generated power is regulated by an MP1584 buck converter, then the TP4056 charging module safely charges the LiPo battery. The LiPo battery acts as the primary energy storage unit of the system that provides a stable voltage supply to the ESP32 microcontroller and other connected components.

The ESP32 microcontroller serves as the central processing and control unit of the system. It interfaces with environmental sensors, actuators, display modules, and communication interfaces. The DHT11 temperature and humidity sensor continuously monitors the atmospheric conditions inside the greenhouse, while the soil moisture sensor measures the moisture content of the soil. These sensors transmit real-time data to the ESP32 for processing and decision-making operations.

Based on the sensor readings, the microcontroller automatically controls the connected actuators. When the humidity level falls below the predefined threshold value or, the ESP32 activates the mist maker to maintain suitable humidity conditions.Also, when the temperature levels rise to the threshold, the mist maker still turns on to regulate the temperature unless humidity values are exceeded. Similarly, when the soil moisture level becomes insufficient, the controller turns on the water pump to irrigate the plants.

The processed environmental data is displayed locally on the OLED display, allowing users to observe temperature, humidity, and soil moisture values in real time.

For remote monitoring, the ESP32 transmits the collected data to the back-end system through BLE protocol. The back-end database stores and manages the sensor data, which is then accessed through the user dashboard interface. This enables users to remotely monitor greenhouse conditions and observe and also control the operational status of the system.

4.2. Flow Chart

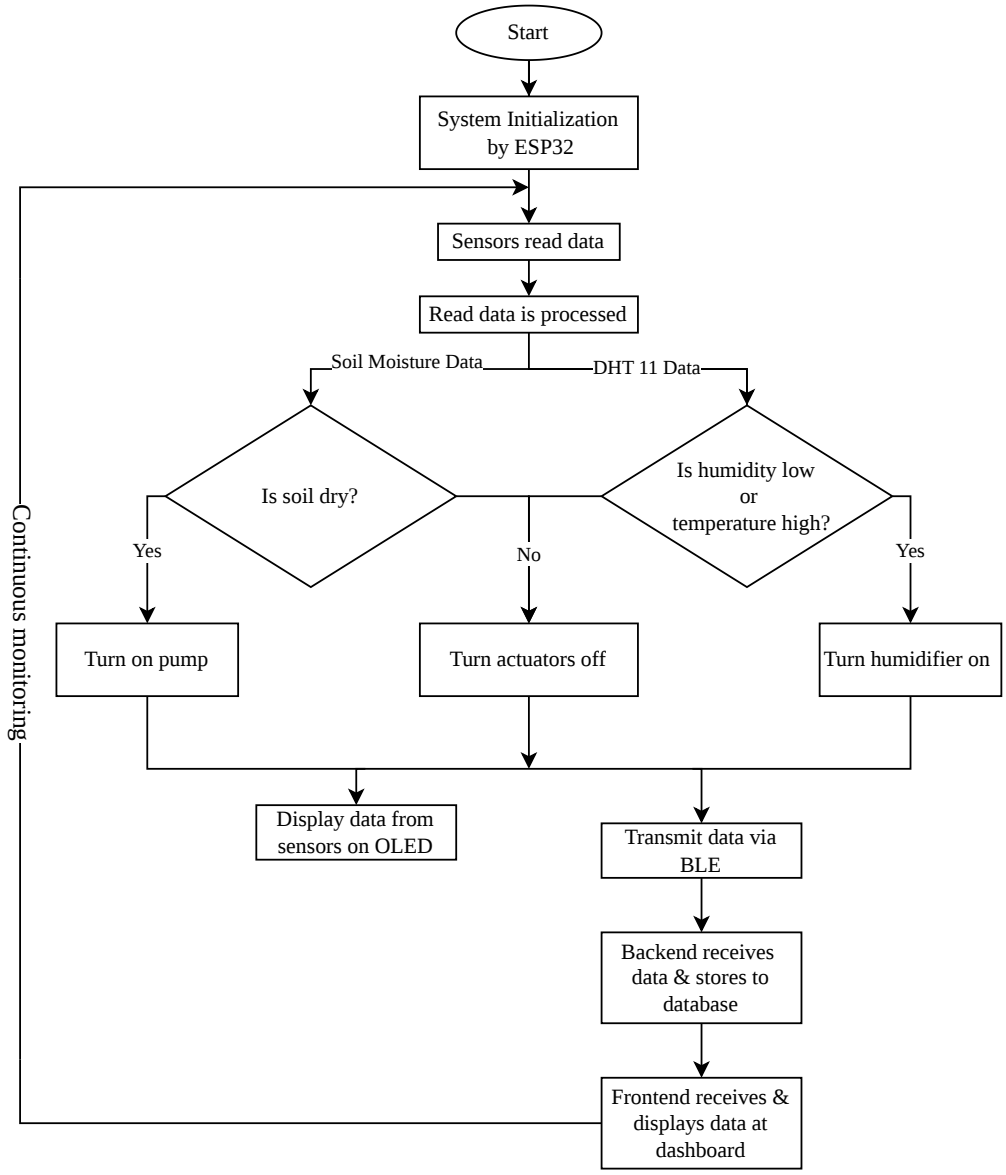

Figure 4.2: Flow Chart of Complete Workflow

4.3. Hardware Setup and System Design

The system is built using a single ESP32 microcontroller that performs environmental data acquisition, processing, decision making, and actuator control.

4.3.1. Sensor Node (Data Acquisition Unit)

The sensor node is responsible for collecting real-time environmental parameters of the greenhouse and transmitting them to the control unit.

- **Function:** Reads sensor data and transmits it
- **Sensors Used:**
  - DHT11 sensor for temperature and humidity measurement
  - Soil moisture sensor for monitoring soil conditions
- **Power Source:** Solar-powered battery system
- **Operation Mode:** Periodic sampling to minimize power consumption

4.3.2. Control Unit (Processing and Actuation Unit)

The control unit acts as the central processing system, receiving sensor data and controlling actuators based on predefined logic.

- **Controller:** ESP32 microcontroller
- **Function:** Processes incoming data and executes control decisions
- **Data Handling:** Compares real-time values with predefined thresholds
- **Actuators Controlled:**
  - Water pump for irrigation
  - Mist Maker for temperature and humidity regulation
- **Output Control:** Each actuator is driven through a dedicated GPIO pin configured as a digital output. The control signals are either ON/OFF or PWM-based depending on the required operation. IRLZ44N MOSFETs are used for switching high-power devices safely.

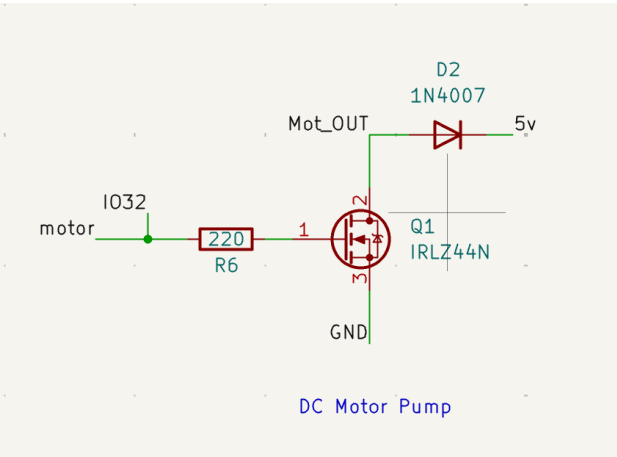

Figure 4.3: DC Pump circuit

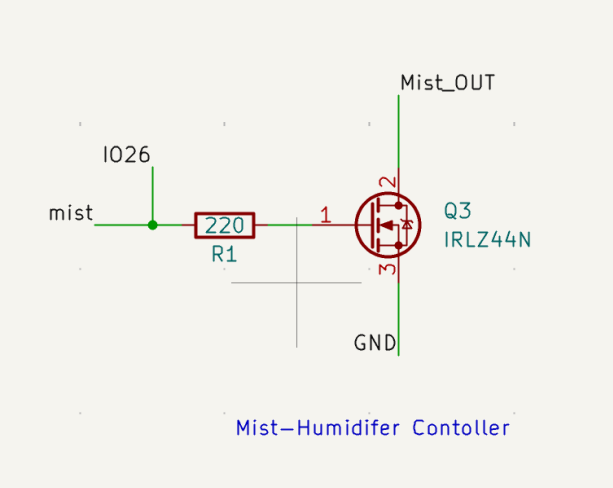

Figure 4.4: Mist Maker circuit

4.3.3. Power Management Unit (Solar Charging System)

The power management unit is responsible for energy generation, storage, regulation, and distribution within the system. It ensures continuous operation of the greenhouse automation system using solar energy.

The overall power flow follows the sequence: solar panel → TP4056 charging module → LiPo battery → MP1584 converter → system components and actuators.

- **Solar Panel (Energy Generation Unit)** The solar panel acts as the primary energy source of the system. It converts solar energy into DC electrical energy, which is used to charge the battery through the charging circuit. The panel is positioned at an optimal angle to maximize sunlight exposure and improve efficiency.

- **TP4056 Charging Module (Battery Charging Unit)** The TP4056 module is responsible for safe and efficient charging of the lithium polymer battery. It operates using a Constant Current–Constant Voltage (CC–CV) charging algorithm, which ensures controlled charging current, prevents overcharging, and protects the battery. It acts as an interface between the solar panel and the energy storage unit.

- **LiPo Battery (Energy Storage Unit)** The 3.7V LiPo battery serves as the energy storage unit of the system. It stores the energy generated by the solar panel and supplies power to the system during periods of low or no sunlight. It acts as a backup and ensures uninterrupted operation of the greenhouse system.

- **MP1584 Buck Converter (Voltage Regulation Unit)** The MP1584 DC-DC buck converter is used for voltage regulation and step-down conversion. It steps down the battery voltage to stable levels required by the ESP32, sensors, and display. It ensures consistent voltage supply and improves system power efficiency.

- **MOSFET Switching Circuit (Load Control Unit)** The IRLZ44N MOSFET is used as an electronic switching device for controlling high-power loads. It is used to control high current devices such as the water pump and mist maker. It enables the ESP32 to safely drive these loads using low power control signals, ensuring efficient and reliable operation.

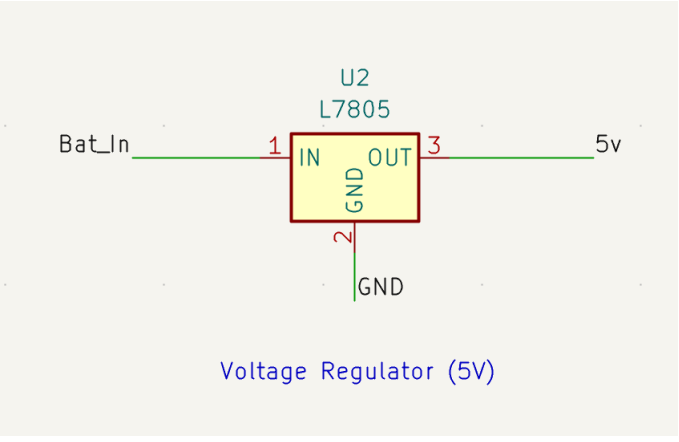

Figure 4.5: Schematic of Voltage Regulator

4.3.4. Power Budget Analysis

The total power consumption of the system is estimated by summing individual component loads under normal operating conditions as per their available datasheets.

S.N.	Component	Voltage (V)	Current (A)	Power (W)
1	DC Water Pump	5	0.2	1.0
2	Mist Maker	5	0.3	1.5
3	ESP32 Controller	3.3	0.5	1.65
4	OLED Display	3.3	0.012	0.04
5	Soil Moisture Sensor	3.3	0.035	0.12
6	DHT11 Sensor	3.3	0.002	0.0066
7	MP1584 (Power management)	4.5 - 28	0.1	0.5
8	TP4056	5	0.05	0.25

Table 4.1: System Power Consumption Table

The DC water pump and mist maker are intermittent loads that operate only occasionally when triggered by environmental conditions. Therefore, they are not included in the continuous power consumption calculation.

Total Continuous Power Consumption:

$$P = 1.65 + 0.04 + 0.12 + 0.0066 + 0.25 \approx 2.07W$$

MOSFET Power Loss (Negligible)

For IRLZ44N, Resistance (R) = 0.025 Ω

$$P = I^2R$$

S.No	Load	Current (A)	Power Loss (W)
1	DC Water Pump	0.2	0.001
2	Mist Maker	0.3	0.00225

Table 4.2: MOSFET Power Loss for Switching Loads

$$P_{\text{MOSFET}} = 0.001 + 0.00225 = 0.00325\,\text{W}$$

Converter Loss (MP1584)

Assuming converter efficiency of 90%, Input power:

$$P_{in} = \frac{2.07}{0.9} = 2.30W$$

Converter loss:

$$P_{loss} = 2.30 - 2.07 = 0.23W$$

Final Calculated Continuous System Power:

Continuous load power = 2.07 W Converter loss = 0.23 W MOSFET loss = 0.00325 W

$$P_{\text{final}} \approx 2.30W$$

4.3.5. Experimental Power and Energy Calculation (Measured Data)

Battery configuration: 2×3.7 V, 1800 mAh in series

Voltage = 7.16 V Capacity = 1.8 Ah

Measured Operating Conditions:

Battery nominal voltage: V = 7.16 V

Initial current (startup condition): $I_1 = 0.11A$

Steady-state current (normal operation): $I_2 = 0.08A$

Startup Power (Transient Condition):

$$P_1 = V \times I_1 = 7.16 \times 0.11 = 0.7876 \approx 0.79W$$

Steady-State Power (Normal Operation):

$$P_2 = V \times I_2 = 7.16 \times 0.08 = 0.5728 \approx 0.57W$$

For energy estimation, steady-state power is considered:

$$P_{avg} \approx 0.57W$$

$$E_{battery} = V \times Ah = 7.16 \times 1.8 = 12.89Wh$$

$$Runtime = \frac{12.89}{0.57} = 22.6 \text{ hours}$$

If V = 7.4V taken, Runtime = 23.3 hours

Experimental Validation

A real-time evaluation was conducted over a duration of 1.5 hours. During this period, the battery voltage decreased from 7.4 V to 7.2 V under normal system operation.

The voltage drop rate is:

$$\frac{7.4-7.2}{1.5} = 0.133 V/hour$$

Considering the usable voltage range the LiPo battery:

- Full charge voltage: 8.4 V

- Cutoff voltage: 6.0 V

- Usable range: 2.4 V

The estimated runtime is:

$$t = \frac{2.4}{0.133} \approx 18 hours$$

The experimental results show that the system consumes significantly lower power (0.57 W) in steady-state operation compared to the theoretical full-load condition (2.30 W). Based on the steady-state power consumption, the system achieves an estimated runtime of approximately 22.6 hours on a single full battery charge.

However, the experimentally estimated runtime was found to be approximately 18 hours during real-time operation. The deviation of approximately 20 - 25% is attributable to actuator switching loads and real-world operating variations.

Despite this difference, the experimental estimations don't vary much with theoretical assumptions, confirming that the system operates within the expected energy efficiency range.

4.3.6. Battery Voltage Monitoring and Calibration

To monitor the battery voltage using the ESP32 ADC, a voltage divider circuit was implemented to scale the battery voltage into the measurable range of the ESP32 ADC (0 V – 3.3 V).

Voltage Divider Design

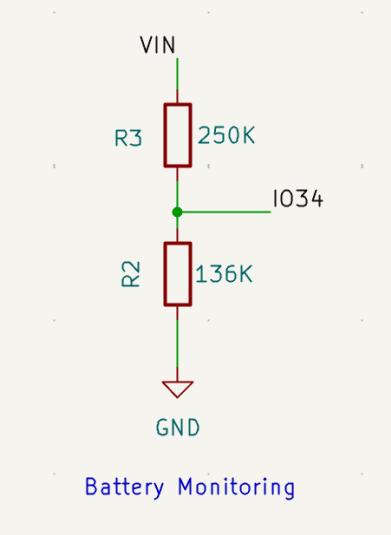

Figure 4.6: Schematic of Voltage Divider

The resistor divider consists of:

$$R_3 = 250\,k\Omega$$

$$R_2 = 136\,k\Omega$$

The output voltage of the divider is given by:

$$V_{out} = \frac{R_2}{R_2 + R_3} \cdot V_{in}$$

Substituting the resistor values:

$$V_{out} = \frac{136}{136 + 250} \cdot V_{in}$$

$$V_{out} = \frac{136}{386} \cdot V_{in}$$

$$V_{out} \approx 0.3523 \cdot V_{in}$$

For a maximum battery voltage of:

$$V_{in} = 8.4V$$

the divider output becomes:

$$V_{out} = 0.3523 \times 8.4$$

$$V_{out} \approx 2.96V$$

which is safely within the ESP32 ADC input range.

ADC Conversion Model

The ESP32 ADC converts the analog voltage into a 12-bit digital value ranging from 0 to 4095.

The ADC conversion equation is:

$$ADC = \frac{V_{out}}{V_{ref}} \times 4095$$

where:

$$V_{ref} = 3.3V$$

Rearranging:

$$V_{out} = \frac{ADC}{4095} \times 3.3$$

Substituting the voltage divider relationship:

$$\frac{ADC}{4095} \times 3.3 = 0.3523 \times V_{in}$$

Solving for V_{in}:

$$V_{in} = \frac{ADC}{4095} \times \frac{3.3}{0.3523}$$

Thus, the battery voltage can be expressed as:

$$V_{in} = ADC \times k_{\text{theoretical}}$$

where:

$$k_{\text{theoretical}} = \frac{3.3}{4095} \times \frac{R_2 + R_3}{R_2}$$

Substituting the resistor values:

$$k_{\text{theoretical}} = \frac{3.3}{4095} \times \frac{386}{136}$$

$$k_{\text{theoretical}} \approx 0.00230$$

Experimental Calibration

To improve measurement accuracy, a single-point experimental calibration was performed.

Measured values:

$$V_{in} = 7.37V$$

$$ADC = 2965$$

The experimental calibration constant is:

$$k_{\text{exp}} = \frac{V_{in}}{ADC}$$

$$k_{\text{exp}} = \frac{7.37}{2965}$$

$$k_{\text{exp}} \approx 0.0024898649$$

This experimentally obtained constant was used in the firmware for battery voltage estimation.

Practical Measurement Observation

During experimental testing, the actual battery voltage measured using a multimeter was:

$$V_{\text{actual}} = 7.37V$$

The voltage measured using the ESP32 ADC fluctuated within the range:

$$V_{\text{measured}} = 7.37V \text{ to } 7.40V$$

The fluctuation was mainly caused by resistor tolerance, ADC noise, reference voltage variation, and minor electrical disturbances in the circuit.

To reduce measurement noise and improve accuracy, multiple ADC samples were taken and averaged in software. A total of:

$$NUM_SAMPLES = 2500$$

samples were collected for each reading, and their average value was used for the

final voltage estimation.

The averaging process significantly improved the measurement stability and reduced random fluctuations in the ADC readings.

Measurement Accuracy and Efficiency

The practical voltage measurement error was very small:

$$E_{\text{voltage}} = |V_{\text{measured}} - V_{\text{actual}}|$$

Considering the maximum observed value:

$$E_{\text{voltage}} = |7.40 - 7.37|$$

$$E_{\text{voltage}} = 0.03V$$

The percentage error is:

$$E_{\%} = \frac{|7.40 - 7.37|}{7.37} \times 100$$

$$E_{\%} \approx 0.41\%$$

Thus, the practical measurement efficiency becomes:

$$\eta = (100 - 0.41)\%$$

$$\eta \approx 99.59\%$$

On another measurement instance, the input voltage was found to be:

$$V_{in} = 7.20V$$

Using the same calibrated system, the measurement was evaluated and the efficiency was found to be:

$$\eta \approx 97.3\%$$

This shows that slight variations in input voltage can occur during different instances, but the system holds a consistent and acceptable accuracy.

Therefore, the implemented battery voltage monitoring system achieved measurement accuracy of approximately above 95% under practical operating conditions.

It should be noted that the above measurement efficiency was obtained under normal monitoring operation of the system, where the ESP32, DHT11 sensor, soil moisture

sensor, and OLED display were active. Under these conditions, the system consumed relatively low power, resulting in highly stable voltage measurements and accuracy above 95%.

However, during actuator operation, additional components such as the mist maker and water pump were activated. These actuators introduced higher current demand and increased the total power consumption of the system to approximately 3 W under peak load conditions. The sudden increase in current draw caused faster battery discharge and temporary voltage drops, which reduced the practical measurement accuracy to approximately 80% in certain instances.

Since the actuators operate only when greenhouse environmental conditions require them and are not continuously active, the core monitoring system remained largely unaffected by them during usual operation. Furthermore, these actuator modules can also be powered using separate power sources if required. Therefore, the above 95% measurement accuracy observed during regular monitoring conditions remains valid for the sensing and battery monitoring operations of the system.

Implementation

The battery voltage reading function implemented in the ESP32 firmware is shown below:

```
#define VOLTAGE_STEP 0.0024898648648649
#define NUM_SAMPLES 2500
float readBatteryVoltage()
{
    unsigned long sum = 0;
    for (int i = 0; i < NUM_SAMPLES; ++i)
    {
        sum += analogRead(BATTERY_ADC_PIN);
    }
    float avg_adc = (float)sum / (float)NUM_SAMPLES;
    return avg_adc * VOLTAGE_STEP;
}
```

The ADC values are averaged over multiple samples to reduce noise and improve voltage measurement stability.

4.3.7. Physical Arrangement of the System

The system components are arranged to simulate real-world conditions:

- Sensor node and control unit are placed within close proximity in the project

setup to minimize loss.

- Clear line-of-sight is maintained for reliable wireless communication.
- DHT11 sensor is positioned at plant level and soil moisture sensor is buried partially for accurate measurements.
- Solar panel is installed at a suitable angle for maximum sunlight exposure.
- Battery and electronics are placed just outside the model.

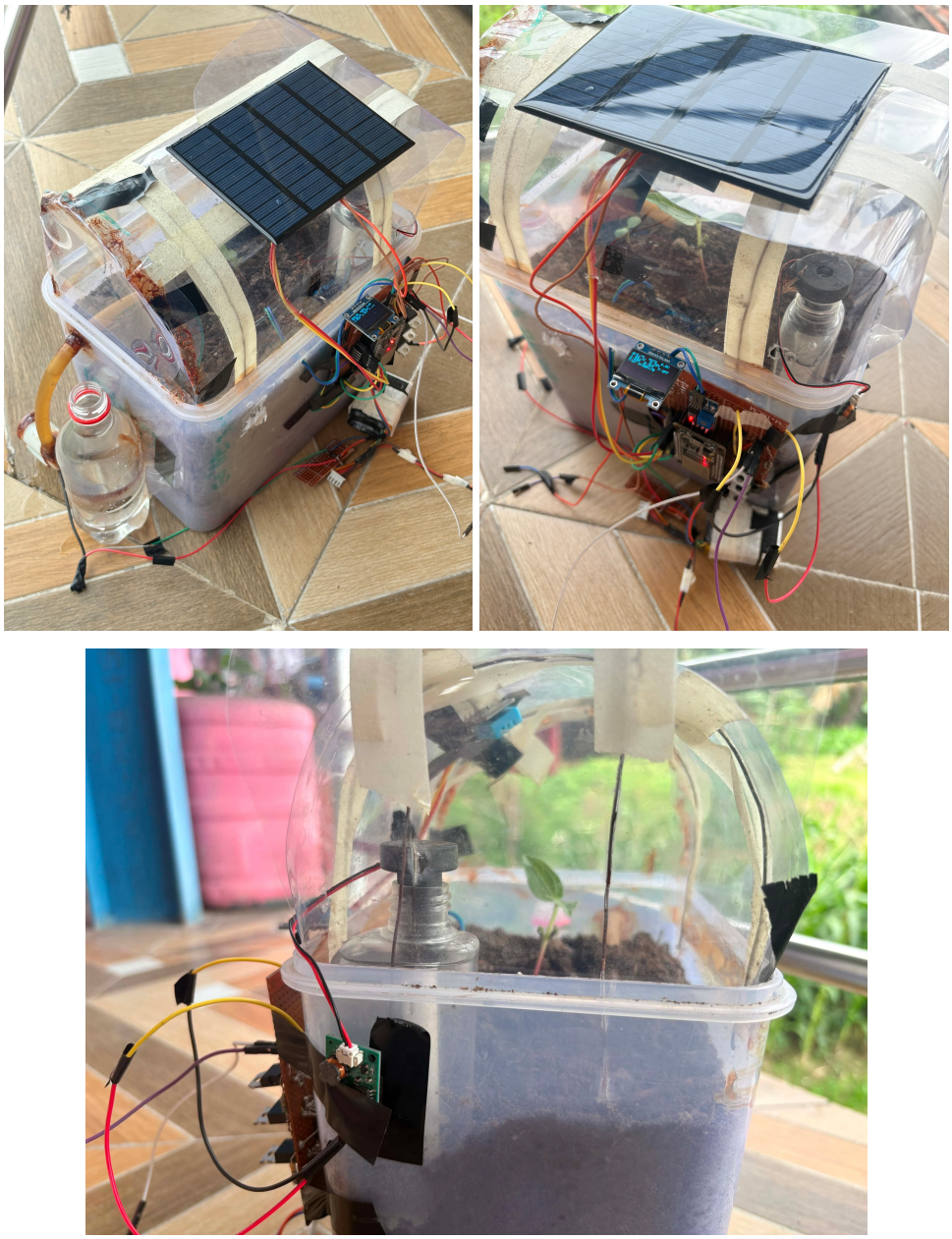

Figure 4.7: Physical Arrangement of Greenhouse

Greenhouse Physical Parameters (Mimic Model)

- Length = 20 cm
- Breadth = 14 cm
- Height of soil = 15 cm
- Height of greenhouse above soil = 13 cm
- Total effective height = 28 cm (with semi-circular top: diameter = 14 cm, height = 13 cm)
- Motor water container (Coca-Cola bottle) = 500 ml
- Mist container (inside greenhouse) = 100 ml

Geometrical and Physical Analysis

The base area of the greenhouse model is:

$$A = 20 \times 14 = 280\,\text{cm}^2$$

The soil volume inside the container is:

$$V_{soil} = 20 \times 14 \times 15 = 4200\,\text{cm}^3$$

The air volume available inside the greenhouse (above soil layer) is:

$$V_{air} = 20 \times 14 \times 13 \approx 3640\,\text{cm}^3$$

The semi-circular top structure improves humidity retention and simulates real greenhouse airflow conditions. The misting system along with this geometry helps maintain a controlled microclimate similar to real agricultural environments.

4.3.8. Pin Configuration and Peripheral Interface

The ESP32 microcontroller acts as the central processing unit of the system, interfacing with all sensors, actuators, and display modules. The system uses GPIO, ADC, and I2C interfaces for hardware communication.

The OLED display is connected using the I2C protocol, which utilizes Serial Data (SDA) and Serial Clock (SCL) lines for synchronous communication. The display operates at the I2C address 0x3C, which is used by the ESP32 for initialization and data transmission.

The pin configuration ensures correct signal routing, stable operation, and prevents hardware conflicts.

S.No	Component	ESP32 Pin	Pin Type	Signal Type
1	DHT11 Sensor	GPIO 5	Bidirectional	Digital (Input)
2	Soil Moisture Sensor	GPIO 34	Input	Analog (ADC)
3	Mist Maker	GPIO 18	Output	Digital (ON/OFF / PWM)
4	Water Pump	GPIO 19	Output	Digital (ON/OFF)
5	OLED SDA	GPIO 21	Bidirectional	I2C Data Line
6	OLED SCL	GPIO 22	Output (Clock)	I2C Clock Line

Table 4.3: ESP32 Pin Configuration and Peripheral Interface

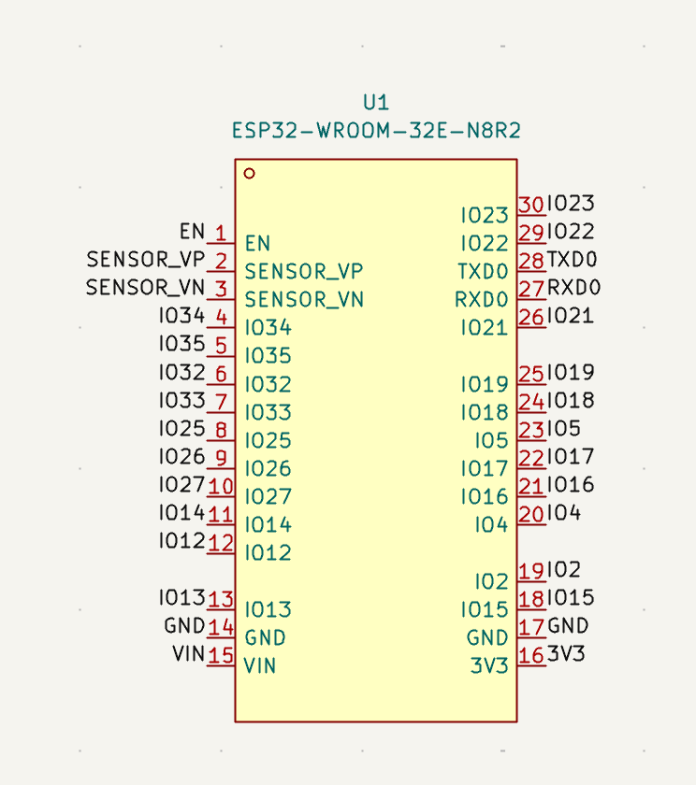

Figure 4.8: ESP32 Pin Configuration

4.4. Data Acquisition and Transmission

The system continuously acquires environmental data using an ESP32 microcontroller. Parameters such as temperature, humidity, and soil moisture are measured at fixed intervals of 1 minute using connected sensors.

The ESP32 processes the collected data internally and applies threshold-based logic for automatic control of actuators such as the water pump and mist maker. The processed data is then formatted and transmitted wirelessly using Bluetooth Low Energy (BLE) for real-time monitoring.

On the receiving side, a BLE-enabled device such as a smartphone or computer captures the transmitted data and displays individual sensor values on the screen for monitoring purposes. This enables low power, wireless, and real-time observation of environmental conditions.

4.5. Data Processing and Monitoring

The system performs all data processing internally using the ESP32 microcontroller. Sensor data from the DHT sensor and soil moisture sensor is read at regular intervals for higher accuracy.

Raw sensor values are validated and converted into meaningful units:

- Temperature (°C)
- Humidity (%)
- Soil moisture (% or ADC value)

The processed data is used directly for:

- Automation decisions
- Wireless transmission via BLE
- Local monitoring

The ESP32 formats the data into structured packets for real-time monitoring.

4.5.1. Real-Time Data Handling

Sensor readings are continuously updated in real time. Invalid values are ignored or corrected. Data is kept lightweight to ensure efficient BLE transmission.

4.5.2. Data Representation

The system data is displayed locally on an OLED interface, showing:

- System mode (AUTO/MAN)
- Temperature (°C)
- Humidity (%)
- Soil moisture (0–100% / 0–4095)
- Mist maker status (ON/OFF)
- Pump status (ON/OFF)
- Threshold values
- Battery voltage

This enables real-time local monitoring without external devices.

4.5.3. Remote Monitoring via BLE

The same processed sensor data is transmitted via Bluetooth Low Energy (BLE) to a user interface (mobile application or web dashboard). The dashboard displays the same parameters that are shown on the OLED interface, ensuring consistency between local and remote monitoring. This allows remote monitoring of environmental conditions in real time without additional requirements.

4.6. Automation Logic and Threshold Calibration

The system uses a threshold based control strategy to automate environmental regulation inside the greenhouse. The control logic continuously compares real-time sensor readings with predefined limits and activates or deactivates actuators accordingly.

- The water pump is activated when soil moisture falls below the lower threshold.
- Mist maker is activated when temperature exceeds the upper threshold or humidity falls below the desired range.
- Sensor values are updated at fixed intervals of 1 minute using the ESP32 microcontroller.

The threshold values are selected based on agronomic requirements for black-eyed pea (cowpea, *Vigna unguiculata*), which grows optimally under controlled temperature and moisture conditions.

Based on these conditions, the system uses:

- Soil moisture threshold: 2200 (Pump ON below this level)
- Temperature threshold: 27°C (Mist ON above this level)
- Humidity threshold: 50% (Mist ON below this level)

The control strategy uses hysteresis i.e, actuators switch OFF only when values return to the normal range, preventing frequent switching and improving system stability. **Actuator Control Implementation**

```
1
2     float temp_high_threshold = 27.0;
3     float temp_low_threshold = 21.0;
4     float humidity_low_threshold = 50.0;
5     float humidity_high_threshold = 70.0;
6     int soil_threshold = 2200;
7
8     // Actuator control logic
9     bool mistOn = false;
10    bool pumpOn = false;
11
12    if (autoMode)
13    {
14        // AUTOMATIC MODE
15        bool mistShouldBeOn = (temperature >
    temp_high_threshold) || (humidity < humidity_low_threshold);
16
17        if (humidity > humidity_high_threshold) {
18            mistShouldBeOn = false;
19        }
20
21        if (mistShouldBeOn)
22        {
23            digitalWrite(MIST_PIN, HIGH);
24            mistOn = true;
25        }
26        else
27        {
28            digitalWrite(MIST_PIN, LOW);
```

```
            mistOn = false;
        }

        if (soilMoistureValue > soil_threshold)
        {
            digitalWrite(PUMP_PIN, HIGH);
            pumpOn = true;
        }
        else
        {
            digitalWrite(PUMP_PIN, LOW);
            pumpOn = false;
        }
    }
    else
    {
        // MANUAL MODE
        if (mistManualOn)
        {
            digitalWrite(MIST_PIN, HIGH);
            mistOn = true;
        }
        else
        {
            digitalWrite(MIST_PIN, LOW);
            mistOn = false;
        }

        if (pumpManualOn)
        {
            digitalWrite(PUMP_PIN, HIGH);
            pumpOn = true;
        }
        else
        {
            digitalWrite(PUMP_PIN, LOW);
            pumpOn = false;
        }
    }
```

4.7. Sensor Data Filtering and Noise Reduction

Raw sensor readings obtained from the DHT11 temperature and humidity sensor are often affected by transient fluctuations, electrical noise, environmental disturbances, and sensor instability. These fluctuations can lead to inaccurate environmental monitoring and unstable actuator triggering within the greenhouse automation system.

To improve measurement stability and enhance the reliability of automation decisions, a moving average filtering technique was implemented in the ESP32 firmware. The filtering algorithm continuously computes the average of the most recent sensor samples using a fixed-size sliding window.

The moving average equation is given by:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i \tag{4.1}$$

where:

- $\bar{x}$ = averaged sensor value
- N = total number of samples
- x_i = individual sensor readings

The implemented algorithm stores temperature and humidity readings inside circular buffer arrays and maintains cumulative sums for efficient real-time averaging. Instead of recalculating the complete summation during every update cycle, the oldest sample is subtracted and the newest sample is added to the running sum. This significantly reduces computational overhead and improves processing efficiency.

The computational complexity of the implemented filtering approach becomes:

$$O(1) \tag{4.2}$$

instead of:

$$O(N) \tag{4.3}$$

which is highly suitable for low-power embedded systems such as the ESP32.

The filtering process improves:

- Sensor stability
- Noise suppression
- Automation reliability
- Actuator response consistency
- Environmental monitoring accuracy

Additionally, invalid sensor readings were rejected before processing. The firmware checks for undefined numerical values using `isnan()` validation to prevent corrupted sensor data from affecting automation logic.

The implemented moving average filtering algorithm is shown below:

```
bool updateDhtAverages(float temperature, float humidity,
                       float &avgTemperature,
                       float &avgHumidity)
{
    if (isnan(temperature) || isnan(humidity))
    {
        return false;
    }

    if (dhtSampleCount < DHT_AVG_SAMPLES)
    {
        dhtTempSamples[dhtSampleIndex] = temperature;
        dhtHumiditySamples[dhtSampleIndex] = humidity;

        dhtTempSum += temperature;
        dhtHumiditySum += humidity;

        dhtSampleCount++;
    }
    else
    {
        dhtTempSum -= dhtTempSamples[dhtSampleIndex];
        dhtHumiditySum -= dhtHumiditySamples[dhtSampleIndex];

        dhtTempSamples[dhtSampleIndex] = temperature;
        dhtHumiditySamples[dhtSampleIndex] = humidity;

```

```
28          dhtTempSum += temperature;
29          dhtHumiditySum += humidity;
30      }
31
32      dhtSampleIndex =
33          (dhtSampleIndex + 1) % DHT_AVG_SAMPLES;
34
35      avgTemperature =
36          dhtTempSum / (float)dhtSampleCount;
37
38      avgHumidity =
39          dhtHumiditySum / (float)dhtSampleCount;
40
41      return true;
42  }
```

Listing 1: DHT11 Moving Average Filtering Algorithm

The filtering mechanism significantly reduced random fluctuations in sensor readings and improved the overall stability of the greenhouse automation system during practical operation.

4.8. BLE-Based Monitoring and Data Management System

The system uses Bluetooth Low Energy (BLE) as the primary communication protocol between the ESP32 microcontroller and the user monitoring device. BLE enables low power, short range wireless transmission of real-time data without the internet.

The ESP32 operates as a BLE server, while the remote monitoring system acts as a BLE client. This architecture ensures efficient and low latency data exchange suitable for a small-scale greenhouse system.

4.8.1. BLE Communication Architecture

The BLE communication framework is designed to support bidirectional data exchange between the greenhouse system and the monitoring interface.

The ESP32 server is responsible for continuously transmitting processed sensor readings and actuator status updates. The client device receives this data and presents it through a user interface for real-time monitoring.

In addition to data transmission, the system supports MANUAL mode optionally, where the client can send control instructions to change actuator states.

Functionality:

- Transmits real-time sensor data
- Provides actuator status updates
- Optionally receives user commands if in Manual mode

4.8.2. User Monitoring Interface

The data received through BLE communication is presented on a user monitoring interface, enabling real-time monitoring and visualization of greenhouse environmental conditions. This interface acts as the visualization layer of the system, translating transmitted sensor data into readable values for user interpretation.

The interface displays key environmental and system parameters received from the ESP32, allowing continuous observation of greenhouse conditions.

Monitored Parameters

- Temperature and humidity values
- Soil moisture level
- Water pump status
- Mist maker status
- Battery level

The interface may include basic visual indicators such as graphical representations to improve readability. It supports both mobile and desktop based applications depending on the deployment environment.

4.8.3. Data Handling and Logging

The system handles data at the client side after receiving structured sensor readings from the ESP32 via BLE communication. The received data is parsed and used for real-time monitoring of greenhouse parameters such as temperature, humidity, soil moisture, and actuator status.

The client device may also store the received data locally, enabling basic logging for future reference and trend analysis.

4.8.4. System Data Flow Architecture

- Sensor nodes collect environmental data.
- Data is transmitted to the ESP32 control unit for processing.
- Automation logic is applied based on thresholds.
- Processed data is transmitted via BLE.
- The user device receives and displays the data.

4.8.5. Monitoring Capability

The system supports short-range monitoring within BLE communication range (theoretically 100m). It enables real-time remote data access with low power consumption and reliable wireless communication without requiring internet.This makes the system suitable for small-scale greenhouse automation applications.

Although the theoretical BLE communication range of the ESP32 is approximately 100 m under ideal conditions, practical testing showed an effective range of about 50-60 m under near line of sight conditions. This reduction is due to environmental interference, signal attenuation, and real world deployment constraints.

4.8.6. Performance Analysis

The BLE communication performance of the system was evaluated using the nRF Connect mobile application. The ESP32 was successfully detected and operated using the configured service UUID `FFF0`, confirming correct implementation of the GATT profile.

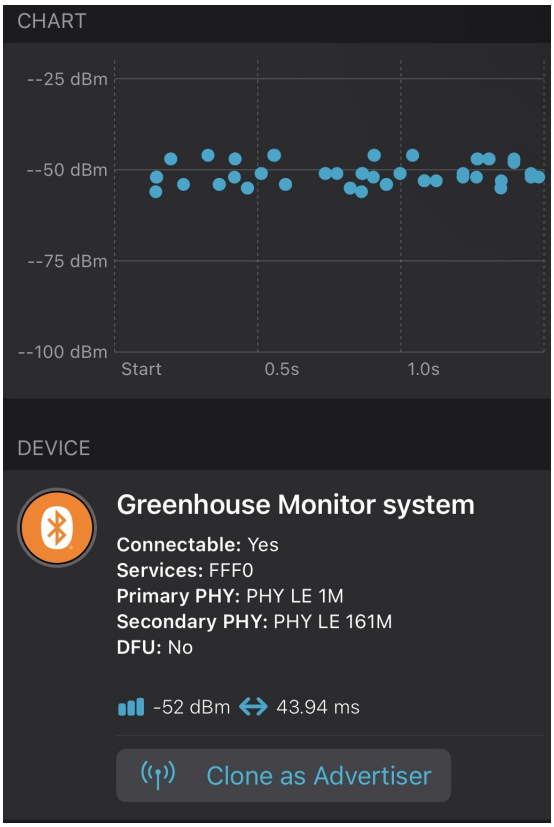

Figure 4.9: RSSI Chart

During testing, the system showed an average RSSI value of approximately -52 dBm, with variations ranging between -47 dBm and -56 dBm across different received packets. The measured connection interval of 43.94 ms indicates low-latency communication, which is suitable for real-time greenhouse monitoring applications.

Overall, the BLE communication system demonstrated stable connectivity, acceptable latency, and reliable data transmission performance. This ensures effective wireless monitoring and control within the intended operational range of the greenhouse system.

4.9. Software System Design

The greenhouse automation system follows a layered architecture consisting of ESP32 firmware, BLE communication, a Node.js gateway, a NestJS backend with PostgreSQL database, and a Next.js dashboard. The system is designed for efficient sensing, communication, processing, and real-time monitoring and control.

4.9.1. System Architecture and Operation

The ESP32 firmware collects environmental data such as temperature, humidity, and soil moisture, and controls actuators like the mist generator and water pump. Data is transmitted to a Node.js gateway using Bluetooth Low Energy (BLE), which forwards it to the backend server.

The backend system developed using NestJS and PostgreSQL processes, validates, and stores sensor data while managing system logic and command execution. The frontend dashboard built with Next.js provides real-time monitoring, analytics, and manual control capabilities.

The complete system is organized into five layers:

- **ESP32 Firmware Layer:** Sensor reading and actuator control
- **BLE Communication Layer:** Wireless data transmission
- **Gateway Layer (Node.js):** Data forwarding and command delivery
- **Backend Layer (NestJS + PostgreSQL):** Data processing and storage
- **Frontend Layer (Next.js Dashboard):** Monitoring and user interaction

4.9.2. Firmware and BLE Communication

Firmware (ESP32)

The firmware runs on the ESP32 and is responsible for sensor reading, actuator control, and BLE communication. It continuously collects data from sensors such as temperature, humidity, and soil moisture, while filtering invalid readings to improve measurement accuracy.

In AUTO mode, the firmware controls actuators according to predefined threshold values. In MANUAL mode, it executes commands received from the backend through BLE communication. The firmware also sends sensor data to the gateway and updates system status information on the OLED display.

BLE Interface

The system implements Bluetooth Low Energy (BLE) using a custom GATT service on the ESP32 to enable communication with the gateway. Sensor data such as temperature, humidity, soil moisture, and battery voltage are transmitted using read/notify characteristics, while control parameters such as operating mode, actuator states, and threshold values are managed through read/write characteristics.

This architecture supports bidirectional communication, allowing both real-time monitoring and remote actuator control. The ESP32 operates as a BLE server, while the gateway acts as a BLE client that subscribes to sensor updates and sends commands when required.

```
// BLE UUIDs
#define SERVICE_UUID "FFF0"

#define SENSOR_CHAR_UUID "FFF1"
// "temp,humid,soil,batteryV"
// Example: "24.5,60.0,2100,7.8"

#define CTRL_CHAR_UUID "FFF4"
// "mode,mist,pump"
// Example: "0,1,0"

#define THRESH_CHAR_UUID "FFF7"
// "tempThresh,soilThresh"
// Example: "25.0,2500"
```

Listing 2: BLE UUID Configuration for ESP32 Communication

This configuration enables efficient bidirectional communication between the ESP32 and gateway for real-time greenhouse monitoring and control.

4.9.3. Backend, Frontend, and System Flow

Backend System

The backend system is developed using NestJS with TypeScript and PostgreSQL. It receives sensor data from the BLE gateway, validates incoming values, processes the information, and stores it in the database for both real-time monitoring and historical analysis.

The backend also manages actuator commands, generates alerts, and exposes REST APIs for communication with both the frontend dashboard and the BLE gateway.

To improve system safety and reliability, the backend applies runtime limits, cooldown periods, command expiration handling, and alert deduplication mechanisms.

Frontend System

The frontend dashboard is built using Next.js, React, and TypeScript. It provides real-time monitoring of environmental conditions and actuator status through a user-friendly interface.

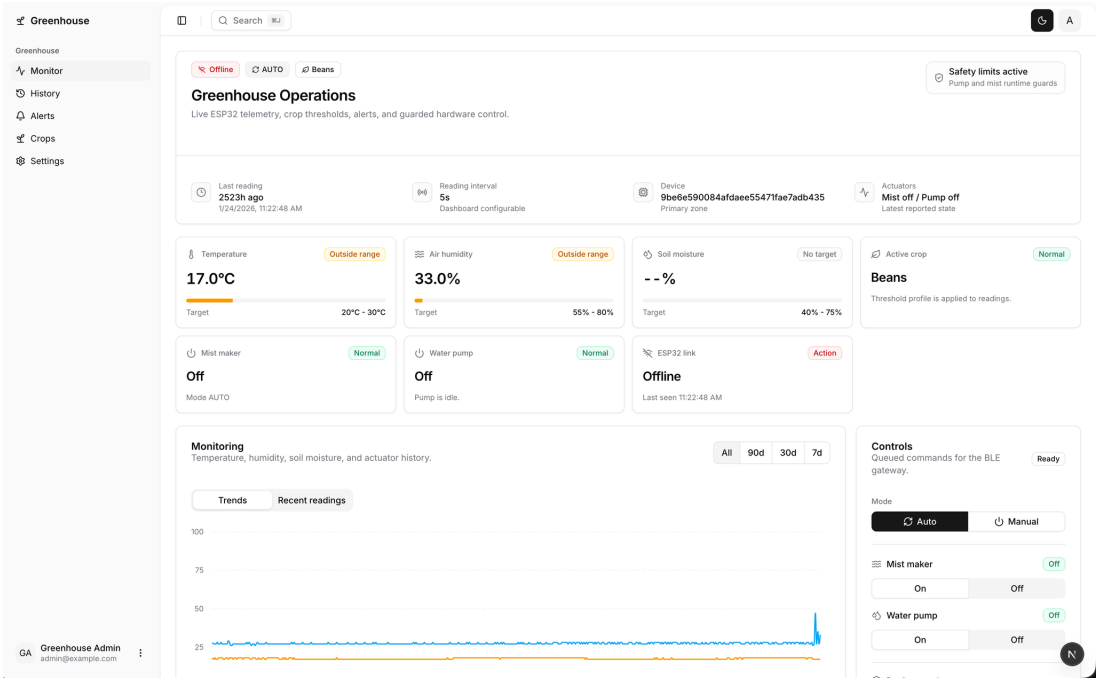

Figure 4.10: Frontend Dashboard

The dashboard displays:

- Temperature and humidity readings
- Soil moisture percentage
- Water pump status
- Mist generator status
- Alert notifications

The frontend periodically fetches updated data from the backend using REST APIs to maintain near real-time monitoring. Authentication and role-based access control are also implemented to ensure secure access to system control functions.

Data Flow Architecture

The system implements bidirectional data flow for monitoring and control operations.

Sensor Data Flow

- ESP32 reads environmental sensor data
- Data is transmitted to the gateway via BLE

- Gateway forwards data to the backend server
- Backend validates and stores data in PostgreSQL
- Dashboard fetches and displays data using REST APIs

Control Command Flow

- User actions from the dashboard are sent to the backend
- Backend creates and manages control commands
- Gateway fetches pending commands from the backend
- Commands are transmitted to ESP32 through BLE write operations
- ESP32 executes commands and updates actuator states

4.9.4. Database, API, and Deployment

The system uses PostgreSQL with Prisma ORM to manage storage of sensor readings, user accounts, actuator states, commands, and alert records.

The backend exposes REST APIs for:

- Sensor data ingestion
- Data retrieval and visualization
- User authentication
- Actuator control and command management

The complete greenhouse automation system is deployed as independent functional components:

- ESP32 firmware for sensing and actuator control
- BLE gateway for wireless communication
- Backend server for processing and management
- PostgreSQL database for persistent storage
- Web dashboard for monitoring and user interaction

Together, these integrated modules provide a reliable and scalable solar-powered greenhouse automation system capable of real-time monitoring, automated environmental control, and remote management.

5. EPILOGUE

5.1. RESULTS AND FINDINGS

A real-time greenhouse automation system was successfully implemented using ESP32 modules, environmental sensors, and automated control mechanisms. By continuously monitoring temperature, humidity, soil moisture, and mist activation conditions, the system efficiently maintained a stable greenhouse environment. Real-time sensor analysis enabled automatic control of irrigation and misting operations, reducing manual effort and optimizing water usage.

5.1.1. Relation in Auto and Manual Mode

The experimental results and performance graphs are demonstrated below:

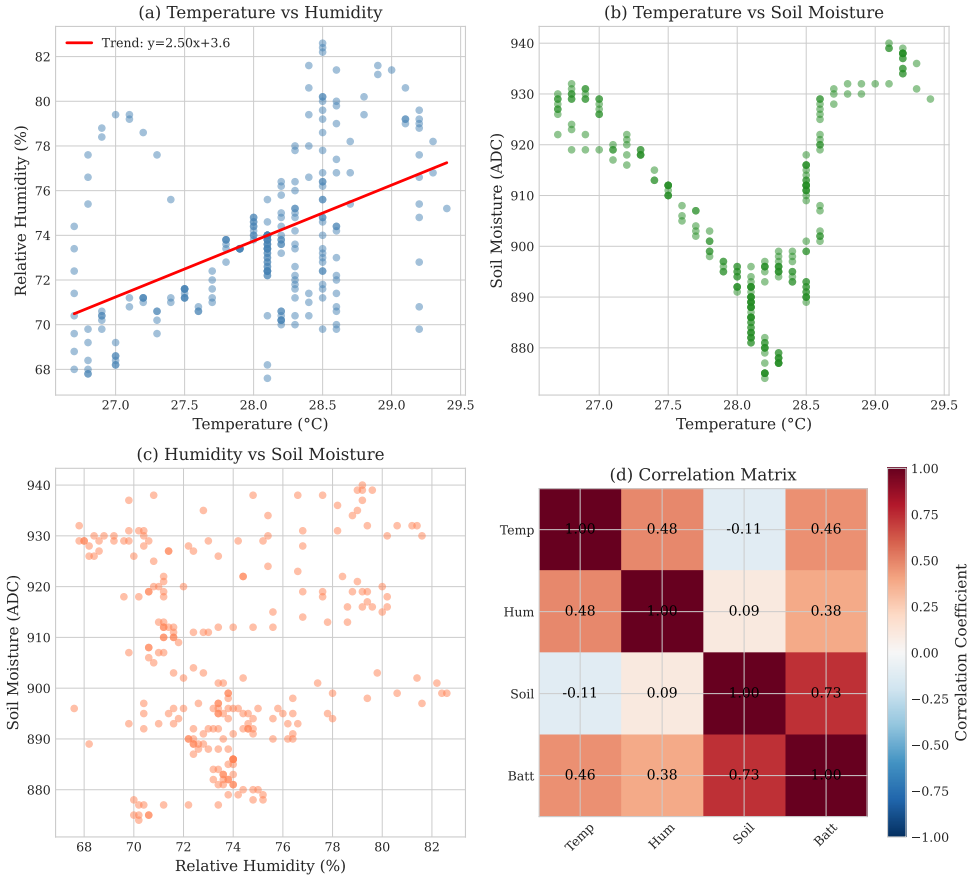

Figure 5.1: Correlation analysis in AUTO Mode

The set of graphs represents the greenhouse system operating in automatic mode. In this mode, the controller automatically adjusts irrigation and environmental con-

ditions based on sensor readings.

The Temperature vs Humidity graph shows a positive relationship between temperature and humidity. As temperature increases, humidity also slightly increases. The trend line indicates stable environmental control inside the greenhouse.

The Temperature vs Soil Moisture graph shows that soil moisture changes gradually with temperature. The moisture values remain within a controlled range, indicating that the automatic irrigation system maintains proper water levels.

The Humidity vs Soil Moisture graph shows a scattered but balanced distribution, meaning both humidity and soil moisture are regulated automatically without sudden fluctuations.

The Correlation Matrix confirms moderate positive relationships among temperature, humidity, soil moisture, and battery parameters. The values indicate that the system maintains environmental balance efficiently.

Overall, the graphs demonstrate that the AUTO mode provides stable and controlled greenhouse conditions, improving plant growth and reducing unnecessary water usage.

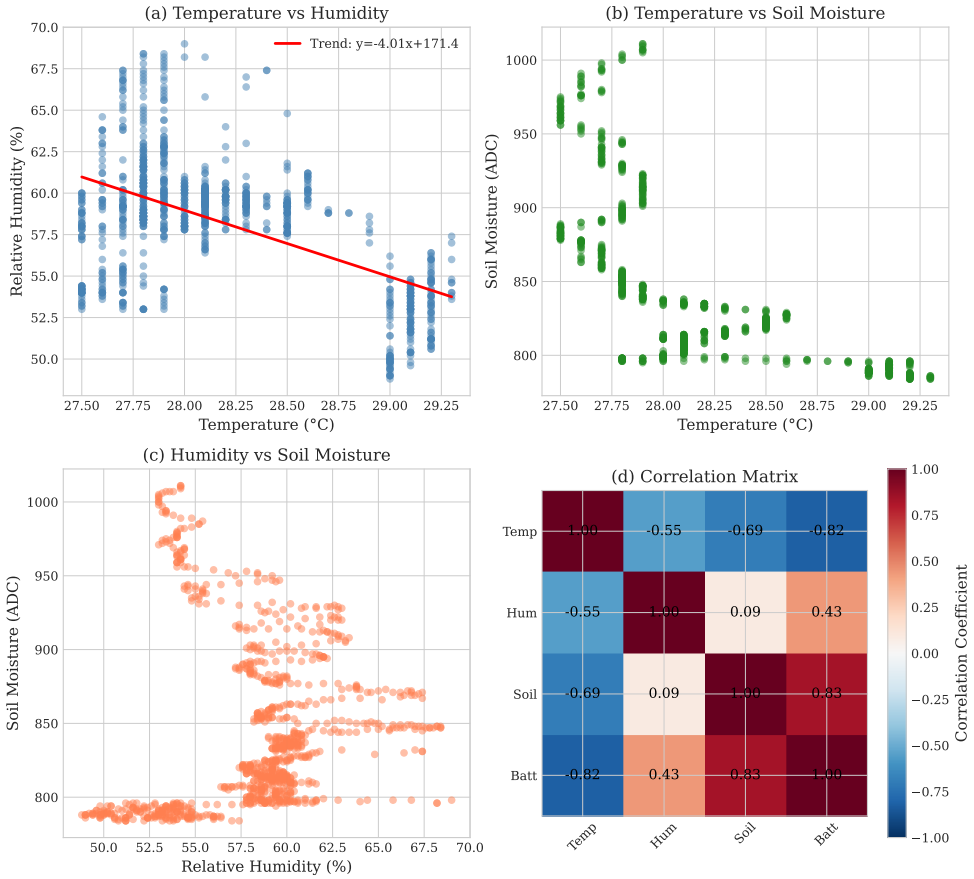

Figure 5.2: Correlation analysis in MANUAL Mode

The set of graphs represents the greenhouse system operating in MANUAL mode. In this mode, irrigation and environmental control depend on human operation instead of automatic sensor-based decisions.

The Temperature vs Humidity graph shows a negative relationship between temperature and humidity. As temperature increases, humidity decreases, indicating less stable environmental control.

The Temperature vs Soil Moisture graph shows large variations in soil moisture with temperature changes. This indicates inconsistent watering and delayed manual response.

The Humidity vs Soil Moisture graph also shows uneven distribution and fluctuations, meaning soil conditions are not maintained uniformly.

The Correlation Matrix shows strong negative correlations between temperature and soil moisture, as well as temperature and battery values. This suggests that environmental conditions vary significantly during manual operation.

Overall, the MANUAL mode results indicate less stability and larger environmental fluctuations compared to AUTO mode. The system depends heavily on user intervention, which may lead to inefficient water management and inconsistent greenhouse conditions.

Parameter	**AUTO Mode**	**MANUAL Mode**	**Operating Threshold**
Number of Samples	835	1056	–
Temperature (°C)	27.32 ± 0.76	28.17 ± 0.52	21°C – 27°C
Humidity (%)	71.9 ± 3.5	58.1 ± 3.8	50% – 70%
Soil Moisture (ADC)	880 ± 31	854 ± 72	< 2200
Battery Voltage (V)	6.47 ± 0.35	7.19 ± 0.13	–
Mist Maker Activation (%)	11.5%	0.7%	–

Table 5.1: Performance Comparison Between AUTO and MANUAL Operating Modes

Table 5.1 compares the performance of the greenhouse system in AUTO and MANUAL operating modes.

The results show that AUTO mode maintains more stable environmental conditions with controlled temperature, humidity, and soil moisture levels through automatic actuator operation. In contrast, MANUAL mode shows larger fluctuations due to dependence on human intervention.

The mist maker activation percentage is higher in AUTO mode, indicating active environmental regulation whenever required. The soil moisture variation is relatively small in both modes because the soil was already sufficiently moist during testing, resulting in minimal water pump activation. The battery voltage remains fairly stable in both modes, with only minor variations due to differences in actuator usage and load conditions.

Overall, the AUTO mode provides better environmental stability and more efficient greenhouse operation compared to MANUAL mode.

5.1.2. Environmental Time Series

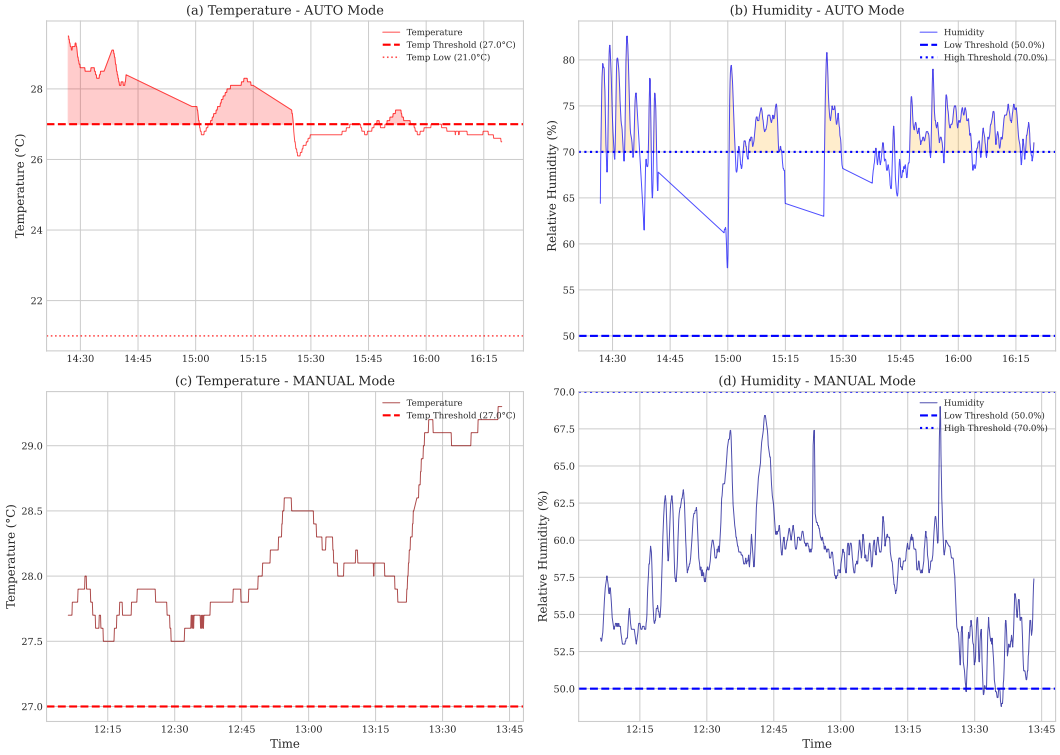

Figure 5.3: Environmental Time Series

The set of graphs illustrates the variation of greenhouse environmental parameters over time under AUTO and MANUAL operating modes, focusing on temperature, humidity, soil moisture, and battery voltage trends.

In AUTO mode, temperature and humidity show controlled fluctuations over time due to continuous feedback-based regulation. Temperature remains within a stable range around the set threshold, while humidity is actively maintained between the defined limits, with only minor short-term variations caused by external environmental disturbances. Soil moisture also remains relatively stable due to automatic irrigation control when required.

In MANUAL mode, environmental parameters show more irregular and less controlled variations over time. Temperature exhibits gradual rises and drops without tight regulation, while humidity fluctuates more frequently outside the desired range due to delayed or inconsistent human intervention. Soil moisture also shows greater variation, indicating less consistent irrigation control.

Overall, the AUTO mode demonstrates smoother and more stable environmental behavior over time, while the MANUAL mode shows higher fluctuations and reduced

control consistency due to reliance on manual operation.

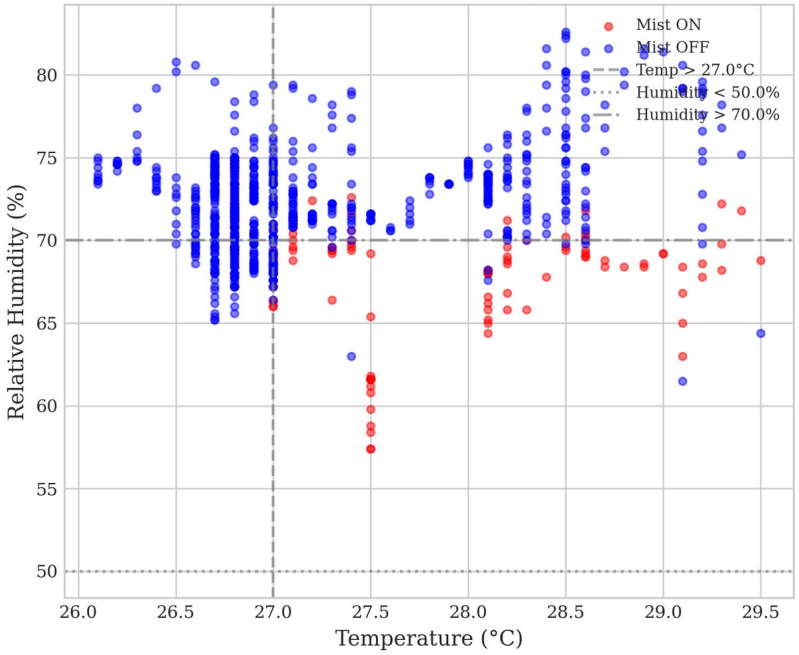

Figure 5.4: Activation Regions of Mist Maker

The graph illustrates the operational behavior of the greenhouse misting system under different temperature and humidity conditions. The red points (**Mist ON**) represent the environmental conditions in which the misting system is activated, while the blue points (**Mist OFF**) indicate conditions where misting is not required.

The misting system is primarily activated when the temperature exceeds 27°C and the humidity level falls below 70%. Under these conditions, the mist maker helps reduce temperature and increase humidity, thereby maintaining a suitable microclimate for plant growth. Conversely, when the humidity level is sufficiently high or the temperature remains within the acceptable range, the misting system stays OFF to prevent excessive moisture accumulation and unnecessary water consumption.

Overall, the figure demonstrates that the automatic mist control mechanism effectively regulates greenhouse environmental conditions while improving water utilization efficiency and maintaining environmental stability.

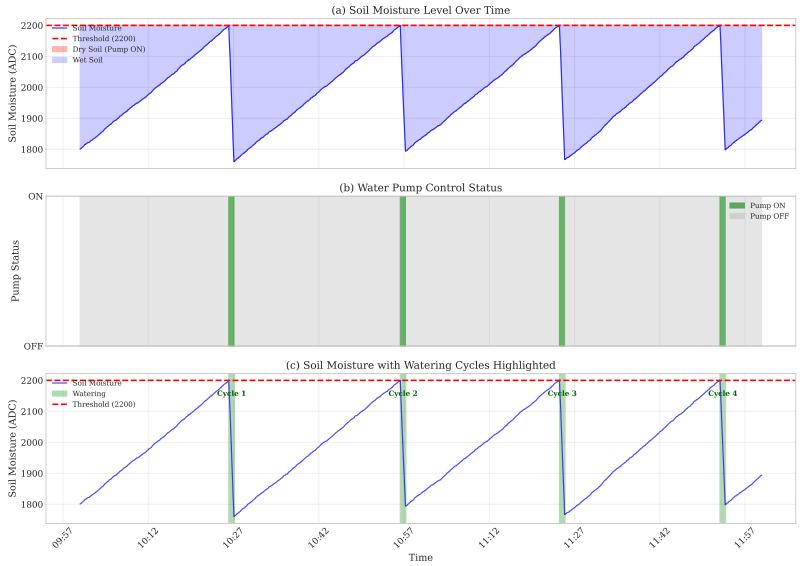

Figure 5.5: Motor Pump ON/OFF

The figure illustrates the automatic soil moisture regulation and pump control behavior of the greenhouse system over time. The upper graph shows the variation of soil moisture levels with respect to the predefined threshold value of 2200 ADC. As the soil gradually dries, the moisture value increases until the threshold is reached.

The middle graph represents the ON/OFF status of the water pump. Whenever the soil moisture reaches the threshold condition, the pump is automatically activated for irrigation. The pump remains ON for approximately 60 seconds during each watering cycle.

The lower graph highlights the complete watering cycles along with the corresponding reduction in soil moisture values after irrigation. Four automatic watering cycles were observed during the experiment. In each cycle, the soil moisture value decreased significantly from approximately 2200 ADC to nearly 1760–1800 ADC after watering.

Overall, the figure demonstrates that the automatic irrigation system successfully detects dry soil conditions, activates the pump at the required threshold, and restores suitable soil moisture levels efficiently without manual intervention.

5.1.3. Solar Charging and Energy Performance Analysis

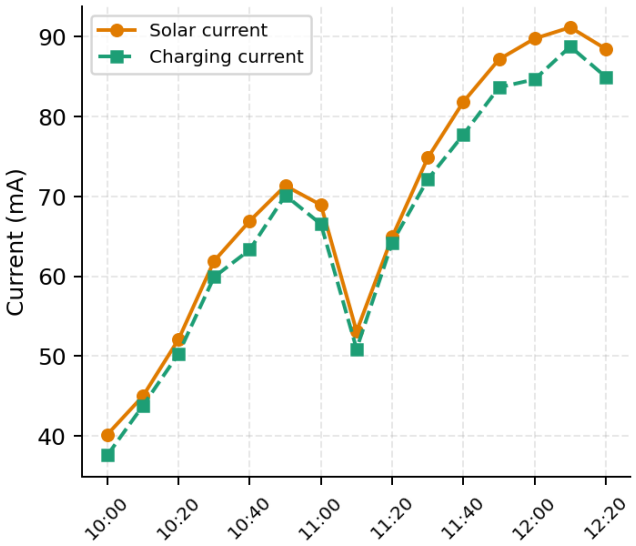

Figure 5.6: Solar Current vs Charging Current

The graph shows the variation of solar current and battery charging current from 10:00 AM to 12:20 PM. Both currents increase gradually as sunlight intensity rises during the day. A temporary drop occurs around 11:10 AM, likely due to reduced sunlight or cloud cover, after which both currents recover and continue increasing.

The solar current reaches a maximum of about 91 mA, while the charging current reaches around 89 mA. The charging current remains slightly lower because of losses in the charging circuit.

Overall, the graph indicates efficient battery charging and stable system performance.

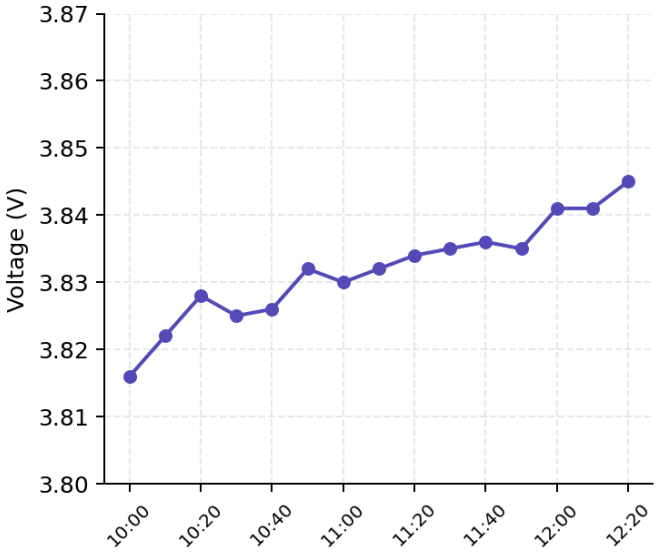

Figure 5.7: Battery Voltage Variation

The battery voltage increases gradually from about 3.816 V to 3.845 V during the observation period, showing continuous battery charging. Small fluctuations appear around 11:00 AM due to temporary variations in solar input.

The steady rise in voltage confirms that the solar charging system is operating properly and storing energy effectively for the greenhouse automation system.

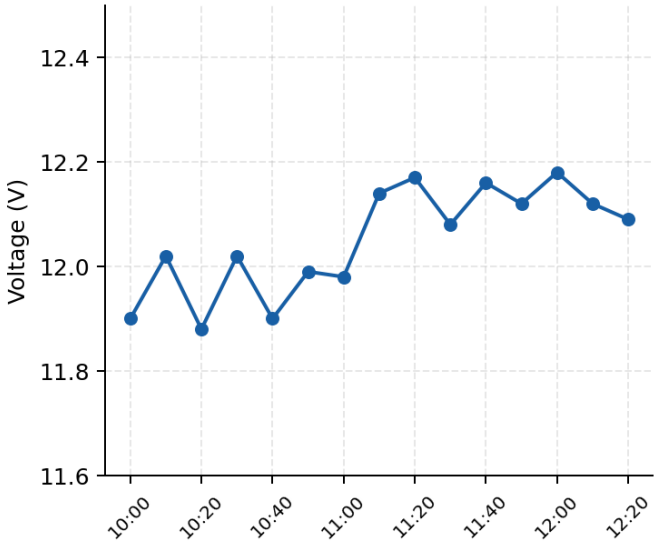

Figure 5.8: Solar Panel Voltage Variation

The solar panel voltage remains fairly stable between 11.9 V and 12.2 V throughout the day. Minor fluctuations occur due to changing sunlight conditions. The voltage slightly increases after 11:00 AM and reaches a maximum of about 12.18 V near

noon.

Overall, the graph demonstrates stable and reliable solar panel performance for powering the greenhouse automation system.

5.1.4. Detection and System Performance

S.No.	Metrics	Value/Item
1	Sensor Accuracy (DHT11)	$\pm 2^{\circ}$C for temperature, $\pm 5\%$ for humidity
2	Sensor Accuracy (Soil Moisture)	95% (correctly identified 'dry' state in 19 out of 20 controlled desiccation tests)
3	Latency (Sensor to Web Dashboard)	Average 2–5 seconds (time for sensor reading to update on the live website)
4	Maximum Sensor Length	Tested successfully up to 100 meters (line-of-sight from the sensors to the central ESP32 node)
5	False Positives (Motor)	The system recorded a 5% failure rate where the motor did not stop and continued running during testing, occurring once during the 20-test sample
6	Energy Performance (Battery Runtime)	22.6 hours operation time using a 7.16 V, 1.8 Ah LiPo battery

Table 5.2: Detection and system performance metrics

5.1.5. Key Findings

- **Self-Powered Operation:** The solar-TP4056 configuration enables 24/7 self-sustained operation in remote locations without reliance on grid power.
- **Power Efficiency:** The MP1584 buck converter maximizes battery life by providing high-efficiency voltage regulation with minimal heat loss.
- **Robust Switching Control:** The IRLZ44N MOSFET allows the low-power ESP32 to safely and efficiently control high-current loads such as water pumps.
- **Environmental Automation Logic:** The combination of DHT11 and soil moisture sensors enables the system to automate actions based on correlated temperature and moisture conditions.

- **Multi-Tasking Reliability:** The ESP32 dual-core architecture ensures real-time sensor monitoring is not interrupted by data transmission or Wi-Fi tasks.
- **System Longevity:** Integrated battery protection circuits prevent overcharging and deep discharge, significantly extending hardware lifespan.

5.2. LIMITATIONS AND FUTURE ENHANCEMENTS

5.2.1. LIMITATIONS

- Battery Charging Constraint: The TP4056 charging module allows only one battery in a series configuration to be charged at a time, limiting charging efficiency for multi-cell setups.
- Weather Dependency: System performance and battery charging are significantly reduced during prolonged periods of cloudy or rainy weather.
- Sensor Degradation: Resistive soil moisture sensors are prone to corrosion over time, which eventually leads to inaccurate data readings.
- Hardware Exposure: Sensitive electronic components remain vulnerable to environmental damage without a high-grade waterproof and dustproof enclosure.
- Limited Communication Range: The current BLE-based communication is suitable only for short-range operation. For large-scale or spatially distributed greenhouses, BLE may not provide sufficient coverage.
- Single-Point Failure: A malfunction in the central ESP32 or the solar charging module causes the entire automation process to stop immediately.

5.2.2. FUTURE ENHANCEMENTS

- Scalable BLE Network: Implementation of multiple BLE sensor nodes distributed across the greenhouse, communicating with a central gateway for scalable and wide-area monitoring.
- Advanced Battery Charging System: Replacement of the TP4056 with a multi-cell battery management system (BMS) or integrated charging architecture to support efficient multi-battery charging and improved energy scalability.

- AI Integration: Use machine learning to analyze weather trends and sensor history, allowing the system to predict watering needs instead of using simple on/off thresholds.
- LoRa Communication: Use LoRa technology to connect multiple devices across several kilometers, suitable for large-scale outdoor deployment.
- Advanced Harvesting: Add an MPPT (Maximum Power Point Tracking) controller to extract maximum power from the solar panel, even under cloudy conditions.
- Mobile Application: Develop a custom mobile application for real-time alerts, remote manual control, and system health monitoring from anywhere.

5.3. CONCLUSION

The greenhouse automation system was successfully designed and implemented using an ESP32-based embedded architecture integrated with environmental sensing, automated actuator control, solar-powered energy management, and wireless monitoring capabilities. The developed system achieved the primary objectives of real-time greenhouse monitoring, automated irrigation and humidity regulation, low-power operation, and remote monitoring through a web-based interface.

The implemented system continuously monitored important environmental parameters including temperature, humidity, soil moisture, and battery voltage using connected sensors and embedded processing. Based on predefined threshold values, the ESP32 automatically controlled the water pump and mist maker, reducing manual intervention and improving environmental stability inside the greenhouse model.

The integration of a solar charging system consisting of a solar panel, TP4056 charging module, LiPo battery, and MP1584 buck converter enabled reliable standalone operation with efficient power management. Experimental measurements showed that the system consumed approximately $0.57W$ under steady-state operating conditions, while the estimated battery runtime reached approximately 22.6 hours on a single charge. The battery voltage monitoring system achieved practical measurement accuracy above 95%, demonstrating reliable energy monitoring performance.

The system also implemented Bluetooth Low Energy (BLE) communication for wireless data transmission between the ESP32 and the gateway system. Sensor data was processed, transmitted, and displayed through a backend and web dashboard

interface developed using Node.js, NestJS, PostgreSQL, and Next.js technologies. This enabled both local and remote monitoring along with manual actuator control capabilities.

Overall, the project demonstrates a practical, sustainable, and scalable solution for smart greenhouse automation. The developed system provides a strong foundation for precision agriculture applications, particularly in small-scale and remote farming environments where low-cost automation and renewable energy integration are important.

References

[1] S. Candy, G. Moore, and P. Freere, "Design and modeling of a greenhouse for a remote region in nepal," *Procedia Engineering*, vol. 49, pp. 152–160, 2012.

[2] M. F. Siddiqui, A. u. R. Khan, N. Kanwal, H. Mehdi, A. Noor, and M. A. Khan, "Automation and monitoring of greenhouse," in *2017 International Conference on Information and Communication Technologies (ICICT)*, 2017, pp. 197–201.

[3] F. Abou-Mehdi-Hassani, A. Zaguia, H. Ait Bouh, and A. Mkhida, "Systematic literature review of smart greenhouse monitoring," *SN Computer Science*, vol. 6, no. 2, p. 95, 2025.

[4] M. Omid and A. Shafaei, "Temperature and relative humidity changes inside greenhouse," *International Agrophysics*, vol. 19, no. 2, pp. 153–158, 2005.

[5] A. Ayuba, "Dht11 based temperature and humidity measuring system," https://www.researchgate.net/publication/376679012_Dht11_Based_Temperature_And_Humidity_Measuring_System, 2023, accessed: 2026-05-01.

[6] K. K. Singh, "Solar energy – the future of energy," pp. 881–884, 2024, accessed: 2026-05-01. [Online]. Available: https://www.ijtsrd.com/papers/ijtsrd67224.pdf

[7] Wikipedia, "Thin-film solar cell," 2026, accessed: 2026-05-01. [Online]. Available: https://en.wikipedia.org/wiki/Thin-film_solar_cell

[8] Lion Care, "Lipo batteries: Properties, advantages and more," 2026, accessed: 2026-05-01. [Online]. Available: http://lion-care.com/en/Info-Center/Industry-knowledge/Overview-of-important-battery-types-and-technologies/LiPo-batteries-properties-advantages-and-more/

[9] Random Nerd Tutorials, "Getting started with the esp32 development board," 2016, accessed: 2026-05-01. [Online]. Available: https://randomnerdtutorials.com/getting-started-with-esp32/

[10] Mouser Electronics, "Dht11 technical data sheet," 2026, accessed: 2026-05-01. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[11] HandsOnTec, “Soil moisture sensor datasheet,” 2026, accessed: 2026-05-01. [Online]. Available: https://handsontec.com/dataspecs/sensor/Soil%20Moisture%20Sensor.pdf

[12] Ingco, “What is a water pump,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.ingco.com/blog/what-is-water-pump

[13] Piezo Ultrasonic, “Ultrasonic mist fogger a015,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.piezo-ultrasonic.com/ultrasonic-mist-fogger-a015.html

[14] OLED-Info, “Oled introduction,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.oled-info.com/oled-introduction

[15] Scribd, “Tp4056 english documentation,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.scribd.com/document/117369667/TP4056-English-Documentation

[16] Monolithic Power Systems, “Mp1584en datasheet,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.monolithicpower.com

[17] Infineon Technologies, “Irlz44n mosfet datasheet,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.infineon.com

[18] PlatformIO, “What is platformio?” 2025, accessed: 2026-05-01. [Online]. Available: https://docs.platformio.org/en/latest/what-is-platformio.html

[19] KiCad Project, “Introduction to kicad,” 2022, accessed: 2026-05-01. [Online]. Available: https://docs.kicad.org

[20] NovelBits, “Bluetooth low energy (ble) complete guide,” 2026, accessed: 2026-05-01. [Online]. Available: https://novelbits.io/bluetooth-low-energy-ble-complete-guide/

[21] Last Minute Engineers, “Esp32 bluetooth classic tutorial and ble overview,” 2026, accessed: 2026-05-01. [Online]. Available: https://lastminuteengineers.com/esp32-bluetooth-classic-tutorial/

[22] M. S. Hossain, M. S. Islam, and M. R. I. Sheikh, “Basic design of solar charging circuit,” 2020, accessed: 2026-05-01. [Online]. Available: https://www.researchgate.net/publication/343258617

[23] A. A. e. a. Ghute, “Solar power battery charger,” 2022, accessed: 2026-05-01. [Online]. Available: https://www.ijrar.org/papers/IJRAR22B2582.pdf

[24] L. Rompis and J. Rante, “Solar charging circuit design,” 2018, accessed: 2026-05-01. [Online]. Available: https://journal.unika.ac.id

[25] U.S. Energy Information Administration, “Solar energy explained,” 2025, accessed: 2026-05-01. [Online]. Available: https://www.eia.gov/energyexplained/solar/photovoltaics-and-electricity.php

[26] OSEPP, “Dht11 sensor datasheet,” 2024, accessed: 2026-05-03. [Online]. Available: https://www.alldatasheet.com

[27] Espressif Systems, “Esp32-wroom-32 datasheet,” 2024, accessed: 2026-05-03. [Online]. Available: https://www.alldatasheet.com

[28] MPS, “Mp1584 datasheet,” 2024, accessed: 2026-05-03. [Online]. Available: https://www.alldatasheet.com

[29] IRF, “Irlz44n datasheet,” 2024, accessed: 2026-05-03. [Online]. Available: https://www.alldatasheet.com

[30] Nanjing Integrated, “Tp4056 datasheet,” 2024, accessed: 2026-05-03. [Online]. Available: https://dlnmh9ip6v2uc.cloudfront.net

[31] Scribd, “Soil moisture sensor manual,” 2021, accessed: 2026-05-03. [Online]. Available: https://www.scribd.com

[32] Jimcontent, “Oled display module datasheet,” 2023, accessed: 2026-05-03. [Online]. Available: https://jimcontent.com

[33] onsemi, “1n4007 rectifier diode datasheet,” 2024, accessed: 2026-05-07. [Online]. Available: https://www.alldatasheet.com/datasheet-pdf/pdf/1934502/ONSEMI/1N4007.html

[34] Vercel, “Next.js documentation,” 2026, accessed: 2026-05-03. [Online]. Available: https://nextjs.org/docs/app

[35] NestJS, “Nestjs documentation,” 2026, accessed: 2026-05-03. [Online]. Available: https://docs.nestjs.com

[36] Prisma, “Prisma schema reference,” 2026, accessed: 2026-05-03. [Online]. Available: https://www.prisma.io/docs

[37] PostgreSQL, “Postgresql documentation,” 2026, accessed: 2026-05-03. [Online]. Available: https://www.postgresql.org/docs/16/

[38] TanStack, “React query documentation,” 2026, accessed: 2026-05-03. [Online]. Available: https://tanstack.com/query

[39] Recharts, “Recharts documentation,” 2026, accessed: 2026-05-03. [Online]. Available: https://github.com/recharts/recharts

[40] Bluetooth SIG, “Bluetooth low energy primer,” 2022, accessed: 2026-05-03. [Online]. Available: https://www.bluetooth.com

[41] Docker, “Docker compose documentation,” 2026, accessed: 2026-05-03. [Online]. Available: https://docs.docker.com/compose

[42] NovelBits, “Bluetooth low energy guide,” 2026, accessed: 2026-05-01. [Online]. Available: https://novelbits.io/bluetooth-low-energy-ble-complete-guide/

[43] NCBI PMC, “Ble comprehensive study,” 2026, accessed: 2026-05-01. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC5751532/

[44] ResearchGate, “Ble research study,” 2021, accessed: 2026-05-01. [Online]. Available: https://www.researchgate.net/publication/356241573_A_Comprehensive_Study_of_Bluetooth_Low_Energy